

Computing Pairings on Elliptic Curves with Embedding Degree Two via Biextensions

Yuhao Zheng¹, Jianming Lin^{1*}, Chang-An Zhao^{1,2}

¹School of Mathematics, Sun Yat-sen University, Guangzhou, 510275, Guangdong, China.

²Guangdong Key Laboratory of Information Security Technology, Guangzhou, 510006, Guangdong, China.

*Corresponding author(s). E-mail(s): linjm28@mail2.sysu.edu.cn;
Contributing authors: zhengyh57@mail2.sysu.edu.cn;
zhaochan3@mail.sysu.edu.cn;

Abstract

Bilinear pairings have emerged as a fundamental tool in public-key cryptography, enabling advanced protocols such as Identity-Based Encryption (IBE), short signatures, and zero-knowledge proofs. This paper focuses on optimizing pairing computations on pairing-friendly curves with embedding degree $k = 2$, addressing both theoretical foundations and practical implementations. We propose an optimized double-and-add ladder algorithm that leverages the technique of y -coordinate recovery, achieving superior performance for the Tate pairing on supersingular curves and the Omega pairing on non-supersingular curves. Our method is implemented based on the RELIC cryptographic library, demonstrating significant efficiency improvements over Miller's algorithm. Specifically, it reduces the number of $\mathbb{F}_p$ -multiplications (resp. CPU clock cycles) by 17.53% (resp. 13.58%) for the reduced Tate pairing on SS-1536 and by 12.37% (resp. 8.39%) for the Omega pairing on NSS-1536. This work establishes the first comprehensive implementation framework for cubical-based pairing computations on curves with embedding degree 2 , providing quantified optimizations for practical cryptographic deployment.

Keywords: Pairing computation Supersingular Non-supersingular Biextension
Miller's algorithm

1 Introduction

Over the past two decades, bilinear pairings have established themselves as a cornerstone of modern cryptography through their unique mathematical properties. These constructions have enabled breakthroughs across multiple domains, from early implementations in identity-based encryption (IBE) [5] and compact digital signatures [6], to recent innovations in privacy-preserving protocols such as zero-knowledge proofs [16, 17, 2]. This mathematical framework has resulted in the development of essential cryptographic components previously deemed impractical, opening new possibilities in secure data processing.

The construction of efficient bilinear maps lies at the heart of pairing-based cryptography. A bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate bilinear map, where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are groups of prime order r . The embedding degree k with respect to r is the smallest integer such that r divides $p^k - 1$. The groups $\mathbb{G}_1$ and $\mathbb{G}_2$ are subgroups of an elliptic curve over a finite field $\mathbb{F}_{p^k}$, while $\mathbb{G}_T$ is a subgroup of the multiplicative group $\mathbb{F}_{p^k}^*$.

The study of pairing computations began with the classical Weil and Tate pairing, which required $\log_2 r$ basic Miller iterations. Over time, many cryptographic researchers have made significant strides in optimizing these computations. Early optimizations focused on shortening the Miller loop, with notable contributions from Duursma-Lee [15] and Barreto *et al.* [4], who leveraged η_T reductions for supersingular curves. Hess *et al.* [25] extended these techniques to ordinary (non-supersingular) curves utilizing Frobenius endomorphisms, and proposed the ate pairing. Further advancements, such as those by Lee *et al.* [26], Matsuda *et al.* [29], and Zhao *et al.* [47], continued to reduce the number of Miller iterations, deriving more efficient pairing computations.

A landmark achievement was Vercauteren's introduction of optimal pairings [43], which reduced the number of iterations to $\log_2 r / \varphi(k)$, where φ denotes the Euler's totient function. For curves equipped with efficiently-computable endomorphisms, the efficiency of optimal pairings can be further enhanced by halving the iterations to $\log_2 r / (2\varphi(k))$. The corresponding variants are named as super-optimal pairings. Recent works [34, 19, 14, 13, 28] have concentrated on implementing super-optimal pairings.

Up to now, the most effective algorithm for pairing computations is Miller's algorithm [31]. There are several alternative methods for evaluating the Miller functions, such as elliptic net algorithm (ENA) [41] together with its improved variant IENA [10]. Despite further accelerations by Cai *et al.* [8], (I)ENA is still less efficient than Miller's algorithm. Recently, Robert [35] proposed a novel approach for pairing computation exploiting biextensions [32] via cubical arithmetic. By making an adjustment to the traditional Montgomery ladder, one can obtain simple and efficient algorithms for the Weil and Tate pairing on elliptic curves [33]. This method outperforms Miller's algorithm for many isogeny-based schemes [33]. On the other hand, a comprehensive analysis of biextensions in pairing-based cryptography, along with detailed cost evaluations, was conducted in [27].

Drawing inspiration from the GLV method [22], Scott [38] pioneered the construction of super-optimal pairings on pairing-friendly curves with embedding degree 2.

These curves allow for low Hamming weight group orders [37], and offer optimization potential via denominator elimination [3], leading to efficient Miller loops. They also enable times-2 compression of Tate pairing values [39]. In addition, they permit the replacement of point multiplications with fast exponentiation operations, which benefits some identification protocols such as IBE [5]. On the other hand, these curves provide strong security against index calculus attacks [30, 40] and help reduce computational costs. Besides, the techniques of pairing compression have been effectively utilized to minimize bandwidth requirements in this context [21].

However, there remains a lack of research on utilizing biextension and cubical arithmetic to compute pairings on curves with $k = 2$ in elliptic curve cryptography. This paper delves into the pairing computations on the curves (including supersingular and non-supersingular curves) with $k = 2$ via biextensions, exploring both theoretical foundations and practical implementations to advance the state-of-the-art.

Our contributions. The key contributions of this paper are summarized as follows.

- **Optimized double-and-add ladder.** Combining the technique of y -coordinate recovery [7], we propose a new double-and-add ladder leveraging biextensions distinct from prior approaches in [35, 27, 33], delivering superior performance on both supersingular and non-supersingular curves with embedding degree 2. Our ladder is tailored with the pairing computations where the Miller loop has a low Hamming weight, specifically of the form $2^{e_1} \pm 2^{e_2}(\pm 1)$, $e_1, e_2 \in \mathbb{N}_+$. By employing the group law of biextensions, we re-express the formula as the product of two biextension elements together with a line function. We utilize the technique of y -coordinate recovery for computing the line function. Our method outperforms the compatible addition and three-way addition under this circumstance, allowing for a more efficient pairing computation.
- **Applications for three types of pairings on curves with $k = 2$.** We apply our new ladder on the following three types of pairing computations.
 1. **The Tate pairing on supersingular curves.** Supersingular curves can be used in pairing-based cryptography, such as SS-1536 [36], a supersingular curve prominent in IBE and escrow ElGamal systems [11]. Notice that pairing-friendly supersingular curves with a Hamming weight of 3 for odd r are easy to find, where r is the order of the Tate pairing. Our ladder is highly suitable for pairing computations on such curves.
 2. **The variant of the Tate pairing on non-supersingular curves.** As mentioned in [38], IBE can also be implemented on non-supersingular curves, which are more efficient than the equivalent supersingular curves since the length of the miller loop of them can be shorten by the non-trivial automorphisms [46]. Pairing-friendly non-supersingular curves with $\lambda = 2^{e_1} + 2^{e_2}$ can be easily found even if we want to have much greater control of r , where λ is the pairing loop parameter related to the non-trivial automorphisms. Our ladder is also highly applicable to this case since the Hamming weight of λ is only two.
 3. **The Omega pairing on non-supersingular curves.** Zhao *et al.* proposed the Omega pairing in [45], which is particularly efficient on elliptic curves with $k = 2$.

Since this pairing utilizes automorphisms, its application on non-supersingular curves is considered. We implement our ladder on the Omega pairing computations and conclude that this pairing is the most efficient on non-supersingular curves with $k = 2$.

- **C-implementation and efficiency comparison.** Our algorithms have been integrated into the C-language cryptographic library RELIC [1] on SS-1536 and NSS-1536. Additionally, we make systematic efficiency comparisons against Miller’s algorithm. The experimental results illustrate that our new algorithms derive a 17.53% (resp. 13.58%) reduction in terms of $\mathbb{F}_p$ -multiplications (resp. CPU clock cycles) for the reduced Tate pairing on SS-1536 compared with Miller’s algorithm. For the Omega pairing on NSS-1536, our method achieves a 12.37% (resp. 8.39%) reduction in terms of $\mathbb{F}_p$ -multiplications (resp. CPU clock cycles).

This work establishes the first comprehensive implementation framework for biextension-based pairing computation on both supersingular and non-supersingular curves with $k = 2$, featuring precisely quantified optimizations for practical cryptographic deployment.

Comparison with the previous literature. Biextensions and cubical arithmetic have proven highly effective in accelerating the pairing computations in isogeny-based cryptography. Compared with Miller’s algorithm, these methods significantly enhance pairing computation efficiency, particularly when the pairing loop parameter is a power of 2 [33]. The computation of cubical-based pairing in pairing-based cryptography has also begun to be explored [27], yet considerable potential remains untapped, especially regarding further improvements in computational efficiency.

Inspired by the remarkable performance of biextensions in isogeny-based cryptography, we investigate their applicability in pairing-based cryptography. We identify use cases where the pairing loop value is close to a power of 2, such as in the case of embedding degree 2, where pairing loop values of the form $2^{e_1} + 2^{e_2} (+1)$ are common. In this paper, we conduct detailed formula derivations for cubical-based pairing computation in the case of embedding degree 2, covering the Tate pairing, its variants, and the Omega pairing (which has not been previously addressed in works on cubical-based pairings for $k = 2$). We also implement and analyze the efficiency of our algorithms on two curves suitable for IBE: SS-1536 and NSS-1536, both with $k = 2$. It is important to note that our work is not a straightforward application of existing algorithms for computing cubical-based pairings; rather, we introduce new algorithms specifically optimized for pairings with $k = 2$.

Outline. The remainder of this paper is organized as follows. Section 2 covers preliminaries related to elliptic curves, pairings and biextensions. Our new algorithms for pairing computation and their formula derivations are introduced in Section 3. Section 4 details the algorithm implementation, cost analysis, and efficiency comparison with Miller’s algorithm. Finally, Section 5 concludes the paper.

2 Preliminaries

2.1 Supersingular and non-supersingular curves

Let E be an elliptic curve defined over a finite field $\mathbb{F}_p$, where $p > 5$ is a prime. Denote by $E(\mathbb{F}_p)$ and $\mathcal{O}_E$ the additive group of E over $\mathbb{F}_p$ and the point of infinity, respectively. A curve E is called **supersingular** if and only if the trace of p -power Frobenius endomorphism $\pi : (x, y) \mapsto (x^p, y^p)$ is divided by p [44, Proposition 4.31].

For IBE schemes, Boneh and Franklin [5] recommend these supersingular curves over $\mathbb{F}_p$

$$\begin{aligned} E_1 : y^2 &= x^3 + ax, \text{ where } p \equiv 3 \pmod{4}, \\ E_2 : y^2 &= x^3 + b, \text{ where } p \equiv 2 \pmod{3}. \end{aligned}$$

Scott introduces the **non-supersingular** curves in [38], on which IBE can equally well be implemented. The non-supersingular curves over $\mathbb{F}_p$ are given as follows

$$\begin{aligned} E_3 : y^2 &= x^3 + ax, \text{ where } p \equiv 1 \pmod{4}, \\ E_4 : y^2 &= x^3 + b, \text{ where } p \equiv 1 \pmod{3}. \end{aligned}$$

We consider two distortion maps $\psi_1 : (x, y) \mapsto (-x, \alpha y)$ and $\psi_2 : (x, y) \mapsto (\beta x, y)$ respectively on E_1 and E_2 , where $\alpha = \sqrt{-1}$ and β is a non-trivial cube root of unity, and $\alpha, \beta \in \mathbb{F}_{p^2} \setminus \mathbb{F}_p$.

Similarly, these maps can be applied to E_3 and E_4 , where they become $\psi_3 : (x, y) \mapsto (-x, \alpha y)$ and $\psi_4 : (x, y) \mapsto (\beta x, y)$, acting as non-trivial automorphisms since $\alpha, \beta \in \mathbb{F}_p$.

2.2 Twists

Let r denote a large prime such that $r \nmid \#E(\mathbb{F}_p)$. The smallest positive integer k satisfying $r \mid p^k - 1$ is the embedding degree with respect to r . Denote the r -torsion subgroup of E by $E[r] = \{P \in E \mid [r]P = \mathcal{O}_E\}$ and the group of the r -th roots of unity by μ_r . The three subgroups $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ of order r are defined as follows

$$\begin{aligned} \mathbb{G}_1 &= E[r] \cap \{P \in E \mid \pi(P) = P\} = E(\mathbb{F}_p)[r], \\ \mathbb{G}_2 &= E(\mathbb{F}_{p^k})[r] \cap \{P \in E \mid \pi(P) = [p]P\}, \\ \mathbb{G}_T &= \mu_r. \end{aligned}$$

Let $\text{Aut}(E)$ be the automorphism group of E and $d = \#\text{Aut}(E)$ be the order of the automorphism group. It holds that $d \in \{2, 3, 4, 6\}$ according to [25]. If $d \mid k$, then E is equipped with a degree- d twist E' over $\mathbb{F}_{p^{k/d}}$. From [25, Proposition 1], ϕ is the twisting isomorphism defined as follows

$$\phi : E' \rightarrow E, (x, y) \mapsto (\xi^2 x, \xi^3 y),$$

where $\xi \in \mathbb{F}_{p^k}$. For the curves admitting with degree- d twists, the subgroup $\mathbb{G}_2$ can be represented by

$$\mathbb{G}_2 \cong E'(\mathbb{F}_{p^{k/d}})[r]. \quad (1)$$

For a supersingular curve E , it can be deduced by Hasse's Theorem that the maximum possible of the embedding degree is 2 if $p > 5$. Based on the previous analysis, we assume that $k = 2$, implying that the twisting isomorphisms of E_1 and E_2 are the distortion maps ψ_1 and ψ_2 , respectively. Then by Eq. (1), one can directly derive that

$$\mathbb{G}_2 \cong E_i(\mathbb{F}_p) = \mathbb{G}_1, \text{ where } i = 1, 2.$$

2.3 Bilinear pairings

This subsection introduces the Miller function, the core of the pairing computations, and presents the corresponding definitions of the Tate pairing, Weil pairing, together with their variants.

2.3.1 Miller function

Let $P \in E$ and $n \in \mathbb{Z}$. The Miller function denoted by $f_{n,P}$ is a normalized rational function with divisor

$$\text{div}(f_{n,P}) = n(P) - ([n]P) - (n-1)(\mathcal{O}_E).$$

For $i, j \in \mathbb{Z}$, there is a relationship between $f_{i,P}$, $f_{j,P}$ and $f_{i+j,P}$

$$f_{i+j,P} = f_{i,P} \cdot f_{j,P} \cdot \frac{l_{[i]P, [j]P}}{v_{[i+j]P}}, \quad (2)$$

where $l_{[i]P, [j]P}$ (resp. $v_{[i+j]P}$) represents the line passing through points $[i]P$ and $[j]P$ (resp. the vertical line passing through point $[i+j]P$ and $-[i+j]P$). An effective method for computing $f_{n,P}(Q)$ is the Miller's algorithm [31].

2.3.2 Tate pairing, Weil pairing and their variants

Let $P \in E(\mathbb{F}_{p^k})[r]$ and $Q \in E(\mathbb{F}_{p^k})$, where k denotes the embedding degree with respect to r . The reduced Tate pairing is defined as

$$e : E(\mathbb{F}_{p^k})[r] \times E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k}) \rightarrow \mu_r, (P, Q) \mapsto f_{r,P}(Q)^{\frac{p^k-1}{r}}$$

since the difference can be eliminated by the final exponentiation.

The endomorphisms of E can be used to shorten the Miller loop of the reduced Tate pairing, which results in a variant of the Tate pairing. As discussed in [38], there exists a positive integer λ , known as the pairing loop parameter, such that $\lambda^2 + 1 \equiv 0 \pmod{r}$ (resp. $\lambda^2 + \lambda + 1 \equiv 0 \pmod{r}$) for the curve E_3 (resp. E_4), which can be utilized in the computation of the variant of the Tate pairing. Zhao *et al.* advanced this by introducing the non-trivial automorphism $\psi_3(P) = [\lambda]P$ for $P \in E_3(\mathbb{F}_p)$ and

$\psi_4(P) = [\lambda]P$ for $P \in E_4(\mathbb{F}_p)$ in [46], computing the pairings respectively on E_3 and E_4 as

$$e(P, Q)^m = \left(f_{\lambda, P}(Q)^\lambda \cdot f_{\lambda, P}(\hat{\psi}_3(Q)) \right)^{\frac{p^k-1}{r}} \quad (3)$$

and

$$e(P, Q)^m = \left(f_{\lambda, P}(Q)^{\lambda+1} \cdot f_{\lambda, P}(\hat{\psi}_4(Q)) \cdot l_{\psi_4(P), \hat{\psi}_4(P)}(Q) \right)^{\frac{p^k-1}{r}}, \quad (4)$$

where k is the even embedding degree, and $mr = \lambda^2 + 1$ (resp. $mr = \lambda^2 + \lambda + 1$) for E_3 (resp. E_4).

Assume that $P, Q \in E[r] \subseteq E(\mathbb{F}_{p^k})$ and $P \neq Q$. The Weil pairing [31] is a bilinear map

$$e_w : E[r] \times E[r] \rightarrow \mu_r, \quad (P, Q) \mapsto (-1)^r \frac{f_{r, P}(Q)}{f_{r, Q}(P)}.$$

If k is even, the following power of Weil pairing

$$\hat{e}_w(P, Q) = e_w(P, Q)^{p^{k/2}-1}$$

is also bilinear.

Similar to the Tate pairing, the length of Miller loop of the Weil pairing can be reduced by exploiting endomorphisms. Zhao *et al.* [45] proposed a variant of Weil pairing called Omega pairing on two types of GLV-curves E_3 and E_4 mentioned in Section 2.1. As pointed out in their paper, the implementation will be more competitive with $k = 2$. Therefore, we only focus on this case. For $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, the Omega pairing on E_3 (resp. E_4) with $k = 2$ is defined as follows

$$\omega : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mu_r,$$

$$(P, Q) \mapsto \left(\frac{f_{\lambda, P}(Q)}{f_{\lambda, Q}(P)} \right)^{p-1},$$

where λ is a positive integer such that $\lambda^2 + 1 \equiv 0 \pmod{r}$ (resp. $\lambda^2 + \lambda + 1 \equiv 0 \pmod{r}$). Compared with the original Weil pairing, the Omega pairing reduces the length of Miller loop to approximately $\log_2(r)/2$.

2.4 Biextensions and cubical points

This subsection primarily focuses on biextensions over elliptic curves, presenting their definitions and elucidating the relationship between biextension elements and Miller functions. Additionally, we introduce cubical points and establish their connection to biextension elements. This enables us to compute the Miller function using cubical arithmetic, which is employed to compute the values of the cubical points.

Biextensions were initially introduced by Mumford [32]. Grothendieck advanced the theory in [24] and introduces a monodromy interpretation of the Weil-Cartier pairing. Stange extends it to the case of the Tate pairing in [42] and introduces elliptic nets to compute the biextension arithmetic. In [35], Robert offers a framework for studying pairings on abelian varieties and proposes the cubical arithmetic to refine the biextension arithmetic.

Let $\mathcal{O}_E$ be the infinity point on the elliptic curve E and the polar divisor be $(\mathcal{O}_E)$. Denote the biextension associated to this divisor by $X_{(\mathcal{O}_E)}$.

Definition 1 ([35]). Consider the divisor denoted by D_P , defined as $(-P) - (\mathcal{O}_E)$. A biextension element is a tuple $(P, Q, g_{P,Q}) \in X_{(\mathcal{O}_E)}$, where P and Q are points on E and $g_{P,Q}$ is a rational function possessing the divisor $D_{P+Q} + D_{\mathcal{O}_E} - D_P - D_Q$. Consequently, the following relation holds

$$\text{div}(g_{P,Q}) = (-P - Q) + (\mathcal{O}_E) - (-P) - (-Q).$$

The Miller function evaluated at $D_Q = (Q + R) - (R)$, $f_{\ell,P}(D_Q)$ can be expressed using the biextension elements $g_{P,Q}(R)$ and $g_{[\ell]P,Q}(R)$, as shown in [27, Eq. (8)]. We state the result as the following lemma.

Lemma 1. Let $g_{P,Q} \in X_{(\mathcal{O}_E)}$. For any non-negative integer ℓ and $R \in E$, it holds that

$$\frac{g_{[\ell]P,Q}(R)}{g_{P,Q}^\ell(R)} = f_{\ell,P}((Q + R) - (R)) \cdot \frac{v_{[\ell]P}((Q + R) - (R))}{v_P^\ell((Q + R) - (R))},$$

where v_P is the vertical line passing through P .

In the remainder of this paper, we consider the Kummer line $K = E/\langle \pm 1 \rangle$, implying that the biextension we are concerned with becomes $X_{2(\mathcal{O}_E)}$. It should be claimed that $g_{P,Q}$ in $X_{2(\mathcal{O}_E)}$ is different from $g_{P,Q}$ in Definition 1 and Lemma 1 as the polar divisor changes from $(\mathcal{O}_E)$ to $2(\mathcal{O}_E)$. We can obtain two sections that yield the projective coordinates X and Z , which are called coordinates of level 2 and $\text{div}(Z) = 2(\mathcal{O}_E)$.

Specifically, as detailed in [35, Section 4] and [33, Appendix A], a cubical point $\tilde{P}$ of level 2 above P corresponds to a chosen value $Z(\tilde{P}) \in \mathbb{F}_{p^k}^*$, with the x -coordinate of P given by $x_P = X(\tilde{P})/Z(\tilde{P})$. By leveraging the cubical representation and cubical ladder for cubical arithmetic, for $g_{P,Q} \in X_{2(\mathcal{O}_E)}$ and $R \in E$, we have

$$g_{P,Q}(R) = \frac{Z(\widetilde{P+Q+R})Z(\tilde{R})}{Z(\widetilde{P+R})Z(\widetilde{Q+R})},$$

where $\text{div}(Z) = 2(\mathcal{O}_E)$. Combining this with Lemma 1, we obtain the following lemma.

Lemma 2. Let the polar divisor be $2(\mathcal{O}_E)$ and Z be a coordinate of level 2. For any non-negative integer ℓ and $(P, Q) \in E(\mathbb{F}_{p^k})[r] \times E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$, the square of the Miller function evaluated at $D_Q = (Q + R) - (R)$ can be computed as

$$f_{\ell,P}(D_Q)^2 = \frac{Z([\ell]\widetilde{P+Q+R})Z(\tilde{R})}{Z([\ell]\widetilde{P+R})Z(\widetilde{Q+R})} \cdot \left(\frac{Z(\widetilde{P+R})Z(\widetilde{Q+R})}{Z(\widetilde{P+Q+R})Z(\tilde{R})} \right)^\ell \cdot \frac{v_P^{2\ell}(D_Q)}{v_{[\ell]P}^2(D_Q)}, \quad (5)$$

where v_P is the vertical line passing through P .

The coordinates X and Z are computed using cubical arithmetic, which gives rise to some cubical ladders such as those described in [35, Algorithm 4.1/4.2] and [27, Algorithm 2].

3 New Cubical-Based Algorithms for Pairing Computation

In this section, we introduce a new ladder for pairing computation using biextensions and cubical arithmetic. We implement this ladder on both supersingular and non-supersingular curves with $k = 2$ to develop new algorithms. We provide detailed derivations of the formulas for these new algorithms.

When computing the Miller function $f_{\ell,P}(Q)$ where ℓ takes the form $\ell = 2^{e_1} \pm 2^{e_2}(\pm 1)$, our ladder (Algorithm 1) outperforms the cubical ladder (Algorithm 8) and the double-and-add ladder (Algorithm 9) from [35, Algorithm 4.1/4.2]. It also surpasses the double-and-add ladder with compatible addition (Algorithm 11) from [27, Algorithm 2], which includes the compatible addition detailed in Algorithm 10 and adapted from [35, Algorithm 5.7].

Specifically, our new ladder employs y -coordinate recovery techniques from [7], eliminating the need for costly iterative steps when processing bit values of 1, which typically increase computational overhead in previous ladders. The efficiency analysis of the new ladder, in comparison with that of other ladders, is briefly discussed in Section 4.1.

We continue to focus on the Kummer line $K = E/\langle \pm 1 \rangle$, which makes Lemma 2 applicable.

3.1 New ladder

Since the key to computing any pairing lies in evaluating the Miller function, we focus on optimizing it and obtain the following theorem.

Theorem 1. *Let Z be a section with $\text{div}(Z) = 2(\mathcal{O}_E)$ and $Z(\tilde{P})$ be the value of cubical point $\tilde{P}$ of level 2. For non-negative integers ℓ, ℓ_1 and ℓ_2 satisfying $\ell = \ell_1 \pm \ell_2$ and $(P, Q) \in E(\mathbb{F}_{p^k})[r] \times E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$, we obtain*

$$f_{\ell,P}(Q)^2 = M \cdot l_{[\ell_1]P, [\pm \ell_2]P}^2(Q) \cdot \frac{v_P^{2\ell}(Q)}{v_{[\ell_1]P}^2(Q) v_{[\ell_2]P}^{1 \pm 1}(Q) v_{[\ell]P}^2(Q)}, \quad (6)$$

where

$$M = \frac{Z([\ell_1]P + Q)}{Z([\ell_1]P)Z(\tilde{Q})} \cdot \left(\frac{Z([\ell_2]P + Q)}{Z([\ell_2]P)Z(\tilde{Q})} \right)^{\pm 1} \cdot \left(\frac{Z(\tilde{P})Z(\tilde{Q})}{Z(P + Q)} \right)^\ell.$$

If k is even, the vertical functions can be eliminated through the final exponentiation by [18, Lemma 3.3], leading to

$$f_{\ell,P}(Q)^2 = M \cdot l_{[\ell_1]P, [\pm \ell_2]P}^2(Q). \quad (7)$$

Proof. Notice that R in $f_{\ell,P}((Q+R)-(R))$ can be replaced by $\mathcal{O}_E$, and we can use the uniformizer to get the extended value of $Z(\mathcal{O}_E)$, yielding the same result as $f_{\ell,P}(Q)$ after the final exponentiation. The case $\ell = \ell_1 + \ell_2$ follows from Lemma 2 and Eq. (2).

Noting that

$$\begin{aligned}\operatorname{div}(f_{\ell,P}) &= \ell(P) - ([\ell]P) - (\ell - 1)(\mathcal{O}_E), \\ \operatorname{div}(f_{-\ell,P}) &= -\ell(P) - (-[\ell]P) - (-\ell - 1)(\mathcal{O}_E), \\ \operatorname{div}(v_{[\ell]P}) &= ([\ell]P) + ([-\ell]P) - 2(\mathcal{O}_E),\end{aligned}$$

we have

$$f_{-\ell,P}^2 = \frac{1}{v_{[\ell]P}^2 \cdot f_{\ell,P}^2}.$$

By replacing $f_{\ell,P}^2$ with Eq. (5), we obtain

$$f_{-\ell,P}(D_Q)^2 = \frac{Z([\ell]P + R)\widetilde{Z(Q + R)}}{Z([\ell]P + Q + R)\widetilde{Z(R)}} \cdot \left(\frac{Z(P + Q + R)\widetilde{Z(R)}}{Z(P + R)\widetilde{Z(Q + R)}} \right)^\ell \cdot \frac{1}{v_P^{2\ell}(D_Q)}. \quad (8)$$

Substituting ℓ_2 for ℓ in Eq. (8) handles the case $\ell = \ell_1 - \ell_2$. $\square$

By invoking Theorem 1, we can construct our new ladder. Specifically, we operate under the assumption that the x and y coordinates of $[\ell]P$ are known. For Eq. (6), we apply level-2 cubical arithmetic to determine the values of the cubical points in M , and the vertical line functions are obtained by leveraging the computed values and the x -coordinate of $[\ell]P$. To compute the line function $l_{[\ell_1]P, [\pm\ell_2]P}(Q)$, we utilize the y -coordinate recovery technique. With the x -coordinates of $[\ell_1]P$ and $[\ell_2]P$, and the x and y coordinates of $[\ell]P = [\ell_1]P \pm [\ell_2]P$ at hand, we can determine its value. This enables the computation of $f_{\ell,P}(Q)^2$ using Eq. (6).

If we further assume that $\ell_1 = 2^{e_1}$ and $\ell_2 = 2^{e_2}$, we can restrict our focus to the case where the bit value is 0 when utilizing level-2 cubical arithmetic, effectively bypassing the more costly case where the bit value is 1.

The specific implementation details of our ladder will be presented in the upcoming subsection. We provide a comprehensive derivation of the line function recovery and explain the handling of the case where the bit is 0. These details are encapsulated in Algorithm 1. Additionally, the ladder leads to Algorithm 2 for computing the variant of the Tate pairing on non-supersingular curves.

3.2 Supersingular case

For a supersingular curve E , the order r is always an odd prime and $k = 2$. We present the following theorem.

Theorem 2. *Assume that $r = r_1 + r_2 + 1$, where r_1 and r_2 are both positive integers and a supersingular curve E is given by the Weierstrass equation $y^2 = x^3 + ax + b$. Denote the conjugate of $e \in \mathbb{F}_{p^2}$ by $\bar{e}$ and the x -coordinate (resp. y -coordinate) of a*

point P by x_P (resp. y_P). The reduced Tate pairing on it can be expressed as

$$\begin{aligned}
& e(P, Q)^2 \\
&= \left(Z([r_1]P + Q) \widetilde{Z([r_2]P + Q)} \cdot \overline{Z([r_1]P)Z(\tilde{Q})Z([r_2]P)Z(\tilde{Q})} \cdot l_{[r_1]P, [r_2]P}^2(Q) \right)^{\frac{p^2-1}{r}} \\
&= \left(Z([r_1]P + Q) \cdot Z([r_2]P + Q) \cdot l_{[r_1]P, [r_2]P}^2(Q) \right)^{\frac{p^2-1}{r}}.
\end{aligned} \tag{9}$$

The last equality holds if $Z([r_1]P) \cdot Z(\tilde{Q}) \cdot Z([r_2]P) \cdot Z(\tilde{Q}) \in \mathbb{F}_p$ within the cubical arithmetic. The line function can be written as

$$l_{[r_1]P, [r_2]P}(Q) = M_1 \left(2y_P Z_1 Z_2 (y_Q - y_P)(X_2 - x_P Z_2) - T_1(x_Q - x_P) \right), \tag{10}$$

where

$$X_i = X(\widetilde{[r_i]P}), \quad Z_i = Z(\widetilde{[r_i]P}), \quad (i = 1, 2),$$

$$M_1 = (2y_P Z_1 Z_2^2)^{-1}, \quad T_1 = T_{11} - T_{12},$$

$$T_{11} = 2bZ_1 Z_2^2 + (aZ_2 + x_P X_2)(x_P Z_2 + X_2)Z_1,$$

$$T_{12} = X_1(x_P Z_2 - X_2)^2 + 2y_P^2 Z_1 Z_2^2,$$

utilizing the known projective coordinates X and Z , along with y_P and y_Q , instead of depending on the y -coordinates of other points.

Proof. Theorem 1 yields Equation (9), since $f_{r,P}(Q)^{2\frac{p^2-1}{r}} = f_{r_1+r_2,P}(Q)^{2\frac{p^2-1}{r}}$, and the final exponentiation permits replacing the inverse with conjugation.

Notice that $[r_1]P + [r_2]P = -P$. To elaborate $l_{[r_1]P, [r_2]P}(Q)$, we rewrite it as follows

$$l_{[r_1]P, [r_2]P}(Q) = l_{P, [r_2]P}(Q) = y_Q - y_P - \frac{y_{[r_2]P} - y_P}{x_{[r_2]P} - x_P} \cdot (x_Q - x_P). \tag{11}$$

Since $y_{[r_2]P}$ is unknown in this expression, we leverage the known values (x_P, y_P) ,

$x_{[r_1]P} = \frac{X(\widetilde{[r_1]P})}{Z(\widetilde{[r_1]P})}$ and $x_{[r_2]P} = \frac{X(\widetilde{[r_2]P})}{Z(\widetilde{[r_2]P})}$ to recover it. By [7], we can deduce that

$$\begin{aligned}
y_{[r_2]P} &= \frac{2b + (a + x_P \cdot x_{[r_2]P})(x_P + x_{[r_2]P}) - x_{[r_1]P} \cdot (x_P - x_{[r_2]P})^2}{2y_P} \\
&= \frac{2b + (a + x_P \frac{X(\widetilde{[r_2]P})}{Z(\widetilde{[r_2]P})})(x_P + \frac{X(\widetilde{[r_2]P})}{Z(\widetilde{[r_2]P})}) - \frac{X(\widetilde{[r_1]P})}{Z(\widetilde{[r_1]P})} \cdot (x_P - \frac{X(\widetilde{[r_2]P})}{Z(\widetilde{[r_2]P})})^2}{2y_P}.
\end{aligned}$$

Substituting the above expressions into Eq. (11), we can derive Eq. (10). □

The application of Theorem 2 on SS-1536 is detailed in Section 4.2.

3.3 Non-supersingular case

This subsection details the computation of the variant of the Tate pairing and the Omega pairing on non-supersingular curves with even embedding degree k .

3.3.1 Case of variant of Tate pairing

The Miller loop on non-supersingular curves can be shortened due to the non-trivial automorphisms discussed in Section 2.3.2, leading to the variant of the Tate pairing computed on E_3 and E_4 via Eqs. (3) and (4), respectively. We state the following theorem for computing the pairing applying our ladder.

Theorem 3. *Let τ denote the non-trivial automorphism ψ_3 on E_3 and ψ_4 on E_4 . Let $P \in E(\mathbb{F}_{p^k})[r]$ and $Q \in E(\mathbb{F}_{p^k})$, with k being even. Let $\lambda = \lambda_1 + \lambda_2$ be the pairing loop parameter, where λ_1 and λ_2 are positive integers. m is an integer satisfying $mr = \lambda^2 + 1$ (resp. $mr = \lambda^2 + \lambda + 1$) for E_3 (resp. E_4). The variant of the Tate pairing on E_3 and E_4 can be computed as*

$$e(P, Q)^{2m} = (V_0^{\lambda+1} V_1^\lambda V_2 \cdot U_0^\lambda U_1)^{\frac{p^k-1}{r}}, \quad (12)$$

and

$$e(P, Q)^{2m} = \left(V_0^{\lambda+2} V_1^{\lambda+1} V_2 \cdot U_0^{\lambda+1} U_1 l_{\tau(P), \hat{\tau}(P)}^2(Q) \right)^{\frac{p^k-1}{r}}, \quad (13)$$

respectively, where

$$\begin{aligned} U_0 &= Z([\lambda_1]P + Q) \cdot Z([\lambda_2]P + Q) \cdot l_{[\lambda_1]P, [\lambda_2]P}^2(Q), \\ U_1 &= Z([\lambda_1]P + \hat{\tau}(Q)) \cdot Z([\lambda_2]P + \hat{\tau}(Q)) \cdot l_{[\lambda_1]P, [\lambda_2]P}^2(\hat{\tau}(Q)), \\ V_0 &= \overline{Z([\lambda_1]P) \cdot Z([\lambda_2]P)}, \\ V_1 &= \overline{Z(P + Q)^\lambda \cdot Z(\tilde{Q})^2} \cdot \left(Z(\tilde{P}) Z(\tilde{Q}) \right)^\lambda, \\ V_2 &= \overline{Z(P + \hat{\tau}(Q))^\lambda \cdot Z(\hat{\tau}(\tilde{Q}))^2} \cdot \left(Z(\tilde{P}) Z(\hat{\tau}(\tilde{Q})) \right)^\lambda. \end{aligned}$$

The line function in U_0 can be expressed as

$$l_{[\lambda_1]P, [\lambda_2]P}(Q) = M_2 \left(2y_{\tau(P)} Z_1 Z_2 (y_Q + y_{\tau(P)}) (x_{\tau(P)} Z_2 - X_2) - T_2 (x_Q - x_{\tau(P)}) \right), \quad (14)$$

where

$$\begin{aligned} X_i &= X([\lambda_i]P), \quad Z_i = Z([\lambda_i]P), \quad (i = 1, 2) \\ M_2 &= (2y_{\tau(P)}Z_1Z_2^2)^{-1}, \quad T_2 = T_{21} - T_{22}, \\ T_{21} &= 2bZ_1Z_2^2 + (aZ_2 + x_{\tau(P)}X_2)(x_{\tau(P)}Z_2 + X_2)Z_1, \\ T_{22} &= X_1(x_{\tau(P)}Z_2 - X_2)^2 + 2y_{\tau(P)}^2Z_1Z_2^2, \end{aligned}$$

and the line function in U_1 can be obtained by replacing Q in Eq. (14) by $\hat{\tau}(Q)$.

Proof. By applying Theorem 1 to replace the Miller functions in Eqs. (3) and (4), we obtain Eqs. (12) and (13), respectively.

Notice that in Theorem 2, $l_{[r_1]P, [r_2]P}(Q) = l_{P, [r_2]P}(Q)$ and $P + [r_2]P = -[r_1]P$, which is similar to $l_{[\lambda_1]P, [\lambda_2]P}(Q) = l_{-\tau(P), [\lambda_2]P}(Q)$ and $-\tau(P) + [\lambda_2]P = [\lambda_1]P$. Thus we can replace x_P, y_P, r_1 and r_2 in Eq. (10) with $x_{\tau(P)}, -y_{\tau(P)}, \lambda_1$ and λ_2 , respectively, to obtain Eq. (14). $\square$

Section 4.3.1 employs Theorem 3 on NSS-1536 to derive the computation of the variant of the Tate pairing.

3.3.2 Case of Omega pairing

On non-supersingular curves, the Omega pairing is computed as

$$\omega(P, Q) = \left(\frac{f_{\lambda, P}(Q)}{f_{\lambda, Q}(P)} \right)^{p-1},$$

where $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, and λ is the pairing-loop parameter (Section 2.3.2). Using our new ladder, we obtain the following theorem for evaluating $\omega(P, Q)^2$ with $k = 2$.

Theorem 4. *Let $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$ and λ be the pairing loop parameter. Assume that $\lambda = \lambda_1 + \lambda_2$, where λ_1 and λ_2 are both positive integers. Notice that $\omega(P, Q)^2$ retains bilinearity since $\gcd(r, 2) = 1$. We can compute the Omega pairing with $k = 2$ as*

$$\begin{aligned} &\omega(P, Q)^2 \\ &= \left(\left(Z([\lambda_1]P + Q)Z([\lambda_2]P + Q)Z([\lambda_1]Q)Z([\lambda_2]Q)Z(\tilde{P})^2 l_{[\lambda_1]P, [\lambda_2]P}^2(Q) \right) \right. \\ &\quad \cdot \left. \overline{\left(Z([\lambda_1]Q + P)Z([\lambda_2]Q + P)Z([\lambda_1]P)Z([\lambda_2]P)Z(\tilde{Q})^2 l_{[\lambda_1]Q, [\lambda_2]Q}^2(P) \right)} \right)^{p-1}, \end{aligned}$$

where $l_{[\lambda_1]P, [\lambda_2]P}(Q)$ is specified in Eq. (14). With $X([\lambda_i]Q)$ and $Z([\lambda_i]Q)$ denoted by X_i and Z_i ($i = 1, 2$) respectively, $l_{[\lambda_1]Q, [\lambda_2]Q}(P)$ can be expressed as follows

$$l_{[\lambda_1]Q, [\lambda_2]Q}(P) = M_3 \left(2y_{\phi^{-1}(\hat{\tau}(Q))}Z_1Z_2(y_P + y_{\hat{\tau}(Q)})(x_{\hat{\tau}(Q)}Z_2 - X_2) - T_3(x_P - x_{\hat{\tau}(Q)}) \right) \xi^3,$$

where

$$\begin{aligned} M_3 &= (2y_{\phi^{-1}(\hat{\tau}(Q))}Z_1Z_2^2)^{-1}, \quad T_3 = T_{31} - T_{32}, \\ T_{31} &= 2bZ_1Z_2^2 + (aZ_2 + x_{\hat{\tau}(Q)}X_2)(x_{\hat{\tau}(Q)}Z_2 + X_2)Z_1, \\ T_{32} &= X_1(x_{\hat{\tau}(Q)}Z_2 - X_2)^2 + 2y_{\hat{\tau}(Q)}^2Z_1Z_2^2. \end{aligned}$$

Proof. Since

$$\omega(P, Q)^2 = \left(\frac{f_{\lambda, P}(Q)}{f_{\lambda, Q}(P)} \right)^{2(p-1)} = \left(f_{\lambda, P}(Q) \cdot \overline{f_{\lambda, Q}(P)} \right)^{2(p-1)},$$

replacing $f_{\lambda, P}(Q)$ and $f_{\lambda, Q}(P)$ by employing Theorem 1 yields the desired result. Swapping P and Q in Eq. (14) gives $l_{[\lambda_1]Q, [\lambda_2]Q}(P)$. $\square$

By invoking Theorem 4, Section 4.3.2 elaborates on the computation of the Omega pairing on NSS-1536.

4 Implementation and Performance Analysis

In this section, we detail the implementation of pairing computations using the biextension $X_{2\mathcal{O}_E}$ and cubical arithmetic. This approach is applied to both the supersingular curve SS-1536 [36] and a non-supersingular curve with embedding degree 2, which is constructed via the Cocks-Pinch method (see [12] and [20, Theorem 4.1]) and named NSS-1536. We have implemented the pairing computation of these curves based on the RELIC library [1].

Additionally, a concrete cost analysis is provided. We compare the computational costs of our implementation algorithms with those of the classical Miller's algorithm. Specifically, our approach leverages the biextension $X_{2\mathcal{O}_E}$ and cubical arithmetic to optimize pairing computations, offering significant efficiency improvements for both SS-1536 and NSS-1536. Given that we have not modified the exponentiation component, we exclude exponentiation operations from the efficiency comparison.

We first introduce some notations for the efficiency comparison, omitting the calculations of additions over finite fields for simplicity. $\mathbf{M}, \mathbf{S}, \mathbf{I}$ denote the costs of multiplication, squaring, and inversion in $\mathbb{F}_{p^2}$, respectively. $\mathbf{m}, \mathbf{s}, \mathbf{i}$ denote the costs of multiplication, squaring, and inversion in $\mathbb{F}_p$, respectively.

4.1 Comparison with other cubical-based ladders

Cubical-based ladders can be utilized for computing the projective coordinates X and Z in cubical-based pairing computation, as noted in [35, Section 4]. These include the cubical double-and-add ladder [35, Algorithm 4.1], the cubical ladder [35, Algorithm 4.2], and the double-and-add ladder with compatible addition [27, Algorithm 2].

We first compare these three types of ladders. In Table 1, we show the operations required by these three ladders when the bit is 0 and 1. Here, **xADD**, **xDBL**, **cADD** and **ThreeWayAdd** denote x -only differential addition, point doubling, compatible addition, and three-way cubical addition (see [35, Algorithm 5.8]), respectively. The cubical

ladder ensures constant-time execution, but it is typically less efficient than the double-and-add ladder, particularly when the number of bit=0 instances significantly exceeds the number of bit=1 instances.

Table 1 Comparison of three cubical-based ladders

Ladder	bit=0	bit=1
Cubical Ladder (Alg. 8)	2xADD + 1xDBL	2xADD + 1xDBL
Double-and-Add Ladder (Alg. 9)	1xADD + 1xDBL	1ThreeWayAdd
Double-and-Add Ladder with cADD (Alg. 11)	1xADD + 1xDBL	2xADD + 1cADD

We focus on the case where $E : y^2 = x^3 + x$ and $k = 2$ (we will explain this choice in the following subsection), and determine the most efficient ladder among these three ladders. Since the number of iterations where the bit equals 1 can typically be controlled to just one in the ladders, we concentrate on comparing the two double-and-add ladders, specifically considering the case where the bit equals 1. In this case, the cost of **ThreeWayAdd** is $29\mathbf{m} + 2\mathbf{s} + 15\mathbf{M} + 2\mathbf{S}$, while the cost of **cADD** is $13\mathbf{m} + 2\mathbf{s} + 2\mathbf{M} + 2\mathbf{S}$ (see Algorithm 10 for details). The costs of the two **xADD** operations in the double-and-add ladder with **cADD** when bit=1 are $6\mathbf{m} + 1\mathbf{M} + 2\mathbf{S}$ and $3\mathbf{m} + 2\mathbf{s}$, respectively. Therefore, when the bit equals 1, the cost of the double-and-add ladder is $29\mathbf{m} + 2\mathbf{s} + 15\mathbf{M} + 2\mathbf{S}$, while the cost of the double-and-add ladder with **cADD** is $24\mathbf{m} + 6\mathbf{s} + 3\mathbf{M} + 4\mathbf{S}$ (which requires extra operations costing $2\mathbf{m} + 2\mathbf{s}$; see Algorithm 11 for details). Hence, in the case we consider, the double-and-add ladder with **cADD** is the most efficient among them.

We analyze the cost of our new ladder proposed in Section 3.1, the double-and-add ladder with line function (Algorithm 1). It achieves higher efficiency than the double-and-add ladder with **cADD** in the case we consider. When the bit equals 0, its cost is also $1\mathbf{xADD} + 1\mathbf{xDBL}$, the same as other double-and-add ladders. However, the new ladder can avoid handling the bit being 1 by simply computing an iteration where bit = 0 and an additional line function, which only adds an overhead of about $(8\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}) + 15\mathbf{m} = 23\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}$, less than $24\mathbf{m} + 6\mathbf{s} + 3\mathbf{M} + 4\mathbf{S}$.

4.2 Pairing computation on SS-1536

In this subsection, we concentrate on the computation process for evaluating the Miller function in the context of the reduced Tate pairing $e : (P, Q) \mapsto f_{r,P}(Q)^{\frac{p^k-1}{r}}$ on SS-1536. The techniques discussed are also applicable to other supersingular curves.

Given that -1 is a quadratic non-residue as $p \equiv 3 \pmod{4}$, the extension field $\mathbb{F}_{p^2}$ can be constructed as $\mathbb{F}_{p^2} = \mathbb{F}_p[i]/(i^2 + 1)$. Since $\mathbb{G}_1$ and $\mathbb{G}_2$ are isomorphic under the twisting isomorphism $\psi_1 : (x, y) \mapsto (-x, iy)$ as shown in Section 2.2, it follows that $x_Q \in \mathbb{F}_p$.

We first review two algorithms for x -only point doubling **xDBL** and differential addition **xADD** as shown in Algorithms 3 and 4. On SS-1536 $E_{SS-1536} : y^2 = x^3 - 3x$, the computational cost of **xDBL** is $2\mathbf{m} + 3\mathbf{s}$ and that of **xADD** is $8\mathbf{m} + 2\mathbf{S}$.

When $a = 1$, specifically for the curve $E : y^2 = x^3 + x$, the computational costs of $\mathbf{xDBL(a=1)}$ and $\mathbf{xADD(a=1)}$ drop to $2\mathbf{m} + 2\mathbf{s}$ and $6\mathbf{m} + 2\mathbf{S}$ (see Algorithms 5 and 6), saving $1\mathbf{s}$ and $2\mathbf{m}$, respectively. Based on this, we aim to transform the curve SS-1536 into $E_{SSmont-1536} : y^2 = x^3 + x$ for higher efficiency, which we refer to as SSmont-1536.

Noting that $p \equiv 3 \pmod{4}$, we can construct a map $\iota : E_{SS-1536} \rightarrow E_{SSmont-1536}$ by first applying a 2-isogeny with kernel $\langle(0, 0)\rangle$ to $E_{SS-1536}$ and then composing it with an isomorphism with image curve $E_{SSmont-1536}$ as supported by [9, Lemma 2]. We can thus obtain

$$\iota : E_{SS-1536} \rightarrow E_{SSmont-1536}$$

$$(x, y) \mapsto \begin{cases} \left(\left(\frac{1}{12} \right)^{\frac{1}{2}} \frac{y^2}{x^2}, \left(\frac{1}{12} \right)^{\frac{3}{4}} \left(1 + \frac{3}{x^2} \right) y \right) & \text{if } (x, y) \neq (0, 0), \infty \\ \infty & \text{if } (x, y) \in \{(0, 0), \infty\} \end{cases}$$

for $(x, y) \in E_{SS-1536}$. This map is beneficial because the orders of points in $\mathbb{G}_1$ and $\mathbb{G}_2$ on $E_{SS-1536}$ are both r , which is coprime to the degree of 2-isogeny. This implies that the orders of these points remain r on $E_{SSmont-1536}$. We develop this map to create SSmont-1536 instead of merely altering the parameters, since we aim for SSmont-1536 to be compatible with systems that include SS-1536.

By leveraging Theorem 2, we propose a ladder named double-and-add ladder with line function, which outperforms other ladders on SSmont-1536 as discussed in Section 4.1. We describe the computational procedure in Algorithm 1, where $\mathbf{xDBLADD}$, presented in Algorithm 7, combines $\mathbf{xDBL}$ and $\mathbf{xADD}$ to eliminate 2 additions in $\mathbb{F}_p$. The line function $l_{[2^{e_1}]P, [2^{e_2}]P}(Q)$ can be computed as Eq. (10). In our implementation, we set $e_1 = 255$ and $e_2 = 41$ in Algorithm 1, since $r = 2^{255} + 2^{41} + 1$ for SSmont-1536. Table 3 offers an in-depth breakdown of the computational costs related to our method.

Algorithm 1 Double-and-Add ladder with line function on $E : y^2 = x^3 + x$

Input: Points $P = (x_P : 1)$, $Q = (x_Q : 1)$ and $Q + P = (X_{Q+P} : Z_{Q+P})$ on $E/\langle\pm 1\rangle$, where $X_P, X_Q \in \mathbb{F}_p$, and two positive integers e_1, e_2 such that $e_1 > e_2$.

Output: The value of the Miller function $f_{2^{e_1}+2^{e_2}, P}(Q)$.

- 1: $(X_{2P}, Z_{2P}, X_{Q2P}, Z_{Q2P}) \leftarrow (x_P, 1, X_{Q+P}, Z_{Q+P})$
- 2: **for** bit $\leftarrow 0$ **to** $e_2 - 1$ **do**
- 3: $(X_{2P}, Z_{2P}, X_{Q2P}, Z_{Q2P}) \leftarrow \mathbf{xDBLADD}(X_{2P}, Z_{2P}, X_{Q2P}, Z_{Q2P}, x_Q, 1)$
- 4: **end for**
- 5: $(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}) \leftarrow (X_{2P}, Z_{2P}, X_{Q2P}, Z_{Q2P})$
- 6: **for** bit $\leftarrow e_2$ **to** $e_1 - 1$ **do**
- 7: $(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}) \leftarrow \mathbf{xDBLADD}(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}, x_Q, 1)$
- 8: **end for**
- 9: **return** $Z_{Q1P} \cdot Z_{Q2P} \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(Q)$

4.3 Pairing computation on NSS-1536

Prior to detailing the computational procedure, we present the construction rationale for NSS-1536 using the Cocks-Pinch method. We aimed for a non-supersingular curve of the form $y^2 = x^3 + x$ to enable faster ladder computations, which implies setting the CM-discriminant $D = 1$. Following [38], we chose $\lambda = 2^{e_1} + 2^{e_2}$ for the form of λ , as $\lambda = 2^{e_1}$ is impractical. To match the 256-bit r and 1536-bit p of SS-1536, we set $r = \lambda^2 + 1$ and assumed $\lambda = 2^{128} + 2^{e_2}$. This approach quickly yielded a suitable curve, NSS-1536, with $\lambda = 2^{128} + 2^{22}$. The parameters are listed in Appendix A.

In this subsection, we delve into the computation process for evaluating the Miller function within the framework of the variant of the Tate pairing (shown in Eqs. (3) and (4)) and the Omega pairing $\omega : (P, Q) \mapsto \left(\frac{f_{\lambda, P}(Q)}{f_{\lambda, Q}(P)} \right)^{p-1}$ on NSS-1536. The methodologies explored here extend their applicability to other non-supersingular curves as well.

Given that 2 is a quadratic non-residue, the extension field $\mathbb{F}_{p^2}$ can be constructed as $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 2)$. The non-trivial automorphism on NSS-1536 is $\psi_3 : (x, y) \mapsto (-x, \alpha y)$, where α is a non-trivial square root of unity in $\mathbb{F}_p$, as discussed in Section 2.1. The twisting map on NSS-1536 is $\phi : E' \rightarrow E, (x, y) \mapsto (u^2x, u^3y) = (2x, 2uy)$, as described in Section 2.2.

4.3.1 Case of variant of Tate pairing

For the reason we claimed at the outset of this section, we use Theorem 3 to compute the line function. Noting that $x_Q \in \mathbb{F}_p$ by the twisting map and $\lambda = 2^{e_1} + 2^{e_2}$, the computation can be written as below

$$e(P, Q)^{2m} = \left(\left(Z([2^{e_1}]P + Q) \cdot Z([2^{e_2}]P + Q) \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(Q) \right)^{2^{e_1} + 2^{e_2}} \cdot Z([2^{e_1}]P + \hat{\psi}_3(Q)) \cdot Z([2^{e_2}]P + \hat{\psi}_3(Q)) \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(\hat{\psi}_3(Q)) \right)^{\frac{p^k - 1}{r}},$$

which is modified from Eq. (12). We can utilize Algorithm 1 to compute the terms

$$Z([2^{e_1}]P + Q) \cdot Z([2^{e_2}]P + Q) \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(Q)$$

and

$$Z([2^{e_1}]P + \hat{\psi}_3(Q)) \cdot Z([2^{e_2}]P + \hat{\psi}_3(Q)) \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(\hat{\psi}_3(Q)),$$

where the line functions $l_{[2^{e_1}]P, [2^{e_2}]P}(Q)$ and $l_{[2^{e_1}]P, [2^{e_2}]P}(\hat{\psi}_3(Q))$ can be computed using Eq. (14).

A natural question that arises is whether we can compute the exponentiation and the ladder simultaneously, similar to the technique used when computing the Miller function [23]. Specifically, we observe that

$$Z_{Q+[2]P} = X_Q \cdot ((X_P + Z_P)(X_{Q+P} - Z_{Q+P}) - (X_P - Z_P)(X_{Q+P} + Z_{Q+P}))^2$$

and

$$X_{Q+[2]P} = Z_Q \cdot ((X_P + Z_P)(X_{Q+P} - Z_{Q+P}) + (X_P - Z_P)(X_{Q+P} + Z_{Q+P}))^2$$

in Algorithms 6 and 7. Consequently, if we multiply X_{Q+P} and Z_{Q+P} by a constant c , we obtain

$$c^2 X_{Q+[2]P} = X_Q ((X_P + Z_P)(cX_{Q+P} - cZ_{Q+P}) - (X_P - Z_P)(cX_{Q+P} + cZ_{Q+P}))^2$$

and

$$c^2 X_{Q+[2]P} = Z_Q ((X_P + Z_P)(cX_{Q+P} - cZ_{Q+P}) + (X_P - Z_P)(cX_{Q+P} + cZ_{Q+P}))^2.$$

This implies that if we multiply X_{Q+P} and Z_{Q+P} by a constant c at the first iteration, the final $Z_{Q+[2^e]P}$ becomes $c^{2^e} \cdot Z_{Q+[2^e]P}$. Additionally, we can combine this technique with the strategy of storing the values of $X_{[2^e]P}$ and $Z_{[2^e]P}$ at each iteration while computing $Z(\widetilde{[2^e]P + Q})$, noting that these values are computed while evaluating both $Z(\widetilde{[2^e]P + Q})$ and $Z(\widetilde{[2^e]P + \hat{\psi}_3(Q)})$.

We present this combined approach in Algorithm 2. We can compute the reduced Tate pairing by setting $e_1 = 128$ and $e_2 = 22$. The algorithms `xDBLADD_s`, presented in Algorithm 12, and `xADD_s`, presented in Algorithm 13, are modified from Algorithms 7 and 6, respectively. We prefer `xDBLADD_s` over `xDBLADD` as it enables storing $X_{[2^e]P} + Z_{[2^e]P}$ and $X_{[2^e]P} - Z_{[2^e]P}$ instead of $X_{[2^e]P}$ and $Z_{[2^e]P}$ separately, thereby saving two additions in `xDBLADD_s`.

While the specific implementations of `xDBL` and `xADD` may vary for other non-supersingular curves due to differences in curve equations, the techniques described above can still be applied with minor adjustments to the algorithms.

A comprehensive overview of the computational costs tied to our approach is provided in Table 4.

4.3.2 Case of Omega pairing

By Theorem 4, we can compute the Omega pairing $\omega(P, Q)^2$ as follows

$$\left(Z(\widetilde{[\lambda_1]P + Q}) Z(\widetilde{[\lambda_2]P + Q}) l_{[\lambda_1]P, [\lambda_2]P}^2(Q) \overline{Z(\widetilde{[\lambda_1]Q + P}) Z(\widetilde{[\lambda_2]Q + P}) l_{[\lambda_1]Q, [\lambda_2]Q}^2(P)} \right)^{p-1},$$

where $\lambda_1 = 2^{128}$ and $\lambda_2 = 2^{22}$, by utilizing the ladder in Algorithm 1 twice, multiplying one result by the conjugate of the other result, and raising the result to the power of $p - 1$. The computational costs associated with our method are meticulously outlined in Table 5.

4.4 Our algorithms vs. Miller's on (N)SS-1536

This subsection offers a computational analysis of the algorithms from Sections 4.2 and 4.3, comparing them with Miller's algorithm. Since our optimizations do not affect

Algorithm 2 Double-and-Add ladders with line function and storage, computing the exponentiation and the ladder simultaneously

Input: Points $P = (x_P : 1)$, $Q = (x_Q : 1)$, $\hat{\psi}_3(Q) = (x_{\hat{\psi}_3(Q)} : 1)$, $x(Q+P) = (X_{Q+P} : Z_{Q+P})$ and $x(\hat{\psi}_3(Q) + P) = (X_{\hat{\psi}_3(Q)+P} : Z_{\hat{\psi}_3(Q)+P})$ on $E/\langle \pm 1 \rangle$

Output: The value of the Miller function $f_{2^{e_1}+2^{e_2},P}(Q)^{2^{e_1}+2^{e_2}} \cdot f_{2^{e_1}+2^{e_2},P}(\hat{\psi}_3(Q))$

```

1: Initialize two arrays  $s_1$  and  $s_2$  of size  $e_1$ 
2:  $(X_{1P}, Z_{1P}) \leftarrow (x_P, 1)$ 
3:  $(X_{Q1P}, Z_{Q1P}) \leftarrow (X_{Q+P}, Z_{Q+P})$ 
4: for bit  $\leftarrow 0$  to  $e_2 - 1$  do
5:    $(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}, s_1[\text{bit}], s_2[\text{bit}]) \leftarrow \text{xDBLADD\_s}(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}, x_Q)$ 
6: end for
7:  $(X_{2P}, Z_{2P}, X_{Q2P}, Z_{Q2P}) \leftarrow (X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P})$ 
8: for bit  $\leftarrow e_2$  to  $e_1 - 1$  do
9:    $(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}, s_1[\text{bit}], s_2[\text{bit}]) \leftarrow \text{xDBLADD\_s}(X_{1P}, Z_{1P}, X_{Q1P}, Z_{Q1P}, x_Q)$ 
10: end for
11:  $f \leftarrow Z_{Q1P} \cdot Z_{Q2P} \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(Q) \triangleright f = f_{2^{e_1}+2^{e_2},P}(Q)$ 
12:  $(X_{QQ1P}, Z_{QQ1P}) \leftarrow \text{xADD\_s}(s_1[0], s_2[0], X_{\hat{\psi}_3(Q)+P} \cdot f, Z_{\hat{\psi}_3(Q)+P} \cdot f, x_{\hat{\psi}_3(Q)})$ 
13: for bit  $\leftarrow 1$  to  $e_2 - 1$  do
14:    $(X_{QQ1P}, Z_{QQ1P}) \leftarrow \text{xADD\_s}(s_1[\text{bit}], s_2[\text{bit}], X_{QQ1P}, Z_{QQ1P}, x_{\hat{\psi}_3(Q)})$ 
15: end for
16:  $(X_{QQ2P}, Z_{QQ2P}) \leftarrow (X_{QQ1P}, Z_{QQ1P})$ 
17: for bit  $\leftarrow e_2$  to  $e_1 - 1$  do
18:    $(X_{QQ1P}, Z_{QQ1P}) \leftarrow \text{xADD\_s}(s_1[\text{bit}], s_2[\text{bit}], X_{QQ1P}, Z_{QQ1P}, x_{\hat{\psi}_3(Q)})$ 
19: end for
20: return  $Z_{QQ1P} \cdot Z_{QQ2P} \cdot l_{[2^{e_1}]P, [2^{e_2}]P}^2(\hat{\psi}_3(Q))$ 

```

the final exponentiation, we do not account for the computational cost and time of it when comparing efficiency.

Grounded on RELIC [1], we compiled our code with GCC 11.4.0 (-O2) and benchmarked it on an Intel Core i9-12900K fixed at 3.20 GHz with TurboBoost and hyper-threading disabled. Each algorithm was executed 2,000 times, and the average CPU clock cycles were recorded.

We ultimately compile the efficiency comparison between our algorithms and Miller's algorithm in Table 2.

Counting computational costs on each algorithm's optimal curve ensures fairness. For supersingular curves, our algorithm runs on SS_{mont}-1536, while Miller's uses SS-1536, each being the fastest for their curve type. For non-supersingular curves, we use the Cocks-Pinch method to find NSS-1536, optimal for Algorithms 1 and 2 with $a = 1$ and $b = 0$. Using the same method, we construct NSS-1536($a = -3$), defined

Table 2 Efficiency comparison of our algorithms vs. Miller’s algorithm based on $\mathbb{F}_p$ -multiplications and CPU clock cycles, with percentage gains of our algorithms over Miller’s algorithm

Curve	Pairing	Algorithm	Cost	Gain	CPU cycles	Gain
supersingular $[\log_2(p)] = 1536$	reduced Tate	Alg 1 Miller’s Alg.	3589m 4352m	17.53%	8354173.3 9666658.4	13.58%
non-supersingular $[\log_2(p)] = 1536$	reduced Tate	Alg 2 Miller’s Alg.	3100m 3092m	−0.26%	7327704.5 6988450.5	−4.85%
	Omega	Alg 1 twice Miller’s Alg.	3619m 4130m	12.37%	8514128.6 9294306.5	8.39%

by $y^2 = x^3 - 3x$, which allows Miller’s algorithm to compute pairings at its fastest. It shares the same λ and r as NSS-1536. The parameters for this curve are provided in Appendix A.

4.4.1 Reduced Tate pairing on SS-1536/SSmont-1536

For the curve SS-1536, where $r = 2^{255} + 2^{41} + 1$, and given that $f_{r_1+r_2+1,P}(Q)^{\frac{p^2-1}{r}} = f_{r_1+r_2,P}(Q)^{\frac{p^2-1}{r}}$, in the Miller loop of it, there are 254 iterations where the bit is 0 and 1 iteration where the bit is 1. Using Jacobian projective coordinates for the computation, the cost for the former is $7\mathbf{m} + 5\mathbf{s} + 1\mathbf{M} + 1\mathbf{S}$ and for the latter is $18\mathbf{m} + 8\mathbf{s} + 2\mathbf{M} + 1\mathbf{S}$. Therefore, the total cost is $254 \times (7\mathbf{m} + 5\mathbf{s} + 1\mathbf{M} + 1\mathbf{S}) + 1 \times (18\mathbf{m} + 8\mathbf{s} + 2\mathbf{M} + 1\mathbf{S}) = 1796\mathbf{m} + 1278\mathbf{s} + 256\mathbf{M} + 255\mathbf{S}$. Noting that $1\mathbf{s} = 1\mathbf{m}$, $1\mathbf{M} = 3\mathbf{m}$, and $1\mathbf{S} = 2\mathbf{m}$, the total cost is equivalent to $4352\mathbf{m}$.

To analyze the computational cost of the computation on SSmont-1536, we present Table 3, including all procedures of the computation of the reduced Tate pairing using Algorithm 1, and provide explanations for the cost of each function.

Table 3 Computational costs of detailed procedures for the reduced Tate pairing using Algorithm 1 on SSmont-1536

Procedure	Cost	Times
$X(P + Q), Z(P + Q)$	$3\mathbf{m} + 1\mathbf{s}$	1
xDBLADD	$8\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}$	254
xDBLADD_last	$6\mathbf{m} + 2\mathbf{s}$	1
$l_{[2^{255}]P, [2^{41}]P}(Q)$	$13\mathbf{m} + 2\mathbf{s}$	1
$Z'(\widetilde{[2^{255}]P + Q}) \cdot Z'(\widetilde{[2^{41}]P + Q}) \cdot l_{[2^{255}]P, [2^{41}]P}(Q)$	$2\mathbf{M}$	1
Total cost: $2054\mathbf{m} + 513\mathbf{s} + 2\mathbf{M} + 508\mathbf{S}$ (equivalent to $3589\mathbf{m}$)		

We compute $X(P + Q)$ and $Z(P + Q)$ using the expressions

$$\begin{aligned} X(P + Q) &= (x_{\psi_1^{-1}(Q)} - x_P) \cdot (x_P \cdot x_{\psi_1^{-1}(Q)} - 1) - 2 \cdot y_P \cdot y_{\psi_1^{-1}(Q)} \cdot i, \\ Z(P + Q) &= (x_P + x_{\psi_1^{-1}(Q)})^2, \end{aligned}$$

which require $3\mathbf{m} + 1\mathbf{s}$.

For the final iteration of our ladder, we modify the formula to compute $e(P, Q)$ instead of $e(P, Q)^2$. The algorithm named `translate` in [33, Algorithm 4] can achieve the same effect, but it requires that one of the points T being added in the last iteration belongs to $E[2]$. In contrast, we simply omit the two squarings in the final step, since we observe that $x_Q \in \mathbb{F}_p$ and $z_Q = 1$. Specifically, denoting $Z([2^{255}]P + Q)^{\frac{1}{2}}$ and $Z([2^{41}]P + Q)^{\frac{1}{2}}$ as $Z'([2^{255}]P + Q)$ and $Z'([2^{41}]P + Q)$, respectively, we can compute the pairing as

$$e(P, Q) = Z'([2^{255}]P + Q) \cdot Z'([2^{41}]P + Q) \cdot l_{[2^{255}]P, [2^{41}]P}(Q),$$

replacing the original Eq. (9). This approach helps us avoid computing the square of the line function. Moreover, for the last `xDBLADD`, since only the values of $X([2^{255}]P)$, $Z([2^{255}]P)$ and $Z'([2^{255}]P + Q)$ are required and $x_P \in \mathbb{F}_p$, the steps in Lines 12, 15, 17, 18 and 19 of Algorithm 7 can be omitted. We denote this variant as `xDBLADD_last`, and its computational cost is reduced to $6\mathbf{m} + 2\mathbf{s}$, noting that Algorithm 7 indicates the cost of `xDBLADD` is $8\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}$.

Denote $X([2^{255}]P)$, $Z([2^{255}]P)$, $X([2^{41}]P)$, $Z([2^{41}]P)$ by X_{255} , Z_{255} , X_{41} , Z_{41} respectively. The line function $l_{[2^{255}]P, [2^{41}]P}(Q)$ is computed utilizing Eq. (10), and its detailed calculation process is presented as follows

1. $t_0 \leftarrow x_P + x_{\psi_1^{-1}(Q)}$, 2. $t_1 \leftarrow Z_{41} \cdot x_P$, 3. $t_2 \leftarrow X_{41} - t_1$, 4. $t_3 \leftarrow Z_{255} \cdot Z_{41}$,
5. $l_1 \leftarrow y_P \cdot y_{\psi_1^{-1}(Q)}$, 6. $t_4 \leftarrow t_4 \cdot t_3$, 7. $t_4 \leftarrow 2 \cdot t_4$, 8. $t_4 \leftarrow y_P^2$,
9. $l_1 \leftarrow l_1 \cdot t_3$, 10. $l_1 \leftarrow l_1 \cdot t_2$, 11. $l_1 \leftarrow 2 \cdot l_1$, 12. $t_1 \leftarrow t_1 + X_{41}$,
13. $t_1 \leftarrow t_1 \cdot Z_{255}$, 14. $t_5 \leftarrow x_P \cdot X_{41}$, 15. $t_5 \leftarrow t_5 + Z_{41}$, 16. $t_5 \leftarrow t_5 \cdot t_1$,
17. $t_3 \leftarrow t_2^2$, 18. $t_3 \leftarrow t_3 \cdot X_{255}$, 19. $t_1 \leftarrow Z_{41} \cdot t_4$, 20. $t_1 \leftarrow t_3 + t_1$,
21. $t_4 \leftarrow t_4 \cdot t_2$, 22. $t_1 \leftarrow t_5 - t_1$, 23. $t_1 \leftarrow t_1 \cdot t_0$, 24. $l_0 \leftarrow t_1 - t_4$,
25. $l \leftarrow l_0 + l_1 \cdot i$.

The cost is $13\mathbf{m} + 2\mathbf{s}$.

4.4.2 Variant of Tate pairing on NSS-1536/NSS-1536($a = -3$)

The computation of the reduced Tate pairing on NSS-1536($a = -3$) is given by Eq.

$$(3), \text{ which is } e(P, Q)^m = \left(f_{\lambda, P}(Q)^\lambda \cdot f_{\lambda, P}(\hat{\psi}_3(Q)) \right)^{\frac{p^k - 1}{r}}.$$

We analyze its computational cost using Miller's algorithm. From the analysis in [46] and the technique in [23], computing $f_{\lambda, P}(Q)$ requires $7\mathbf{m} + 5\mathbf{s} + 1\mathbf{M} + 1\mathbf{S}$ when

the bit is 0, and $18\mathbf{m} + 8\mathbf{s} + 2\mathbf{M} + 1\mathbf{S}$ when the bit is 1. Then, computing $f_{\lambda^2+1,P}(Q) = f_{\lambda,P}(Q)^\lambda \cdot f_{\lambda,P}(\hat{\psi}_3(Q))$ requires $2\mathbf{m} + 1\mathbf{M} + 1\mathbf{S}$ when the bit is 0, and $2\mathbf{m} + 2\mathbf{M} + 1\mathbf{S}$ when the bit is 1. Given that $\lambda = 2^{128} + 2^{22}$ for NSS-1536 ($a = -3$), the total cost is $127 \times (7\mathbf{m} + 5\mathbf{s} + 1\mathbf{M} + 1\mathbf{S} + 2\mathbf{m} + 1\mathbf{M} + 1\mathbf{S}) + 1 \times (18\mathbf{m} + 8\mathbf{s} + 2\mathbf{M} + 1\mathbf{S} + 2\mathbf{m} + 2\mathbf{M} + 1\mathbf{S}) = 1163\mathbf{m} + 643\mathbf{s} + 258\mathbf{M} + 256\mathbf{S}$, equivalent to $3092\mathbf{m}$.

To examine the computational cost of the computation on NSS-1536, we provide Table 4, which details all procedures of the reduced Tate pairing computation using Algorithm 2.

Table 4 Computational costs of detailed procedures for the variant of the Tate pairing via Algorithm 2 on NSS-1536

Procedure	Cost	Times
$X(P + Q), Z(P + Q), X(P + \hat{\psi}_3(Q))$ and $Z(P + \hat{\psi}_3(Q))$	$5\mathbf{m} + 2\mathbf{s}$	1
xDBLADD_s	$8\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}$	127
xDBLADD_s_last	$6\mathbf{m} + 2\mathbf{s}$	1
xADD_s	$6\mathbf{m} + 2\mathbf{S}$	127
xADD_s_last	$4\mathbf{m}$	1
$l_{[2^{128}]P, [2^{22}]P}(Q)$ and $l_{[2^{128}]P, [2^{22}]P}(\hat{\psi}_3(Q))$	$15\mathbf{m} + 1\mathbf{s}$	1
$f = Z'(\widetilde{[2^{128}]P + Q}) \cdot Z'(\widetilde{[2^{22}]P + Q}) \cdot l_{[2^{128}]P, [2^{22}]P}(Q)$	$2\mathbf{M}$	1
$f \cdot Z_{\hat{\psi}_3(Q)+P}$ and $f \cdot X_{\hat{\psi}_3(Q)+P}$	$2\mathbf{m} + 1\mathbf{M}$	1
$Z'(\widetilde{[2^{128}]P + \hat{\psi}_3(Q)}) \cdot Z'(\widetilde{[2^{22}]P + \hat{\psi}_3(Q)}) \cdot l_{[2^{128}]P, [2^{22}]P}(\hat{\psi}_3(Q))$	$2\mathbf{M}$	1
Total cost: $1810\mathbf{m} + 259\mathbf{s} + 5\mathbf{M} + 508\mathbf{S}$ (equivalent to $3100\mathbf{m}$)		

Similar to Section 4.4.1, we can compute the pairing as

$$e(P, Q)^m = Z'(\widetilde{[2^{128}]P + \hat{\psi}_3(Q)}) \cdot Z'(\widetilde{[2^{22}]P + \hat{\psi}_3(Q)}) \cdot l_{[2^{128}]P, [2^{22}]P}(\hat{\psi}_3(Q)).$$

By omitting certain steps in Algorithms 12 and 13, we can reduce their computational costs. We denote these algorithms by xDBLADD_s_last and xADD_s_last, respectively. Specifically, by removing Lines 14, 17, 21, 19, and 20 from Algorithm 12, and Lines 8, 9, 10, 11, and 12 from Algorithm 13, the costs are respectively reduced to $6\mathbf{m} + 2\mathbf{s}$ and $4\mathbf{m}$.

The line functions $l_{[2^{128}]P, [2^{22}]P}(Q)$ and $l_{[2^{128}]P, [2^{22}]P}(\hat{\psi}_3(Q))$ are evaluated using Eq. (14). Denote $X(\widetilde{[2^{128}]P})$, $Z(\widetilde{[2^{128}]P})$, $X(\widetilde{[2^{22}]P})$, and $Z(\widetilde{[2^{22}]P})$ as X_{128} , Z_{128} , X_{22} , and Z_{22} , respectively. At a cost of $13\mathbf{m} + 1\mathbf{s} + 2\mathbf{m}$, the line functions can be

computed jointly as follows

$$\begin{aligned}
& 1. k_0 \leftarrow x_P \cdot Z_{22}, \quad 2. k_1 \leftarrow X_{22} + k_0, \quad 3. k_2 \leftarrow \alpha \cdot y_P, \quad 4. k_3 \leftarrow k_2 \cdot Z_{22}, \\
& 5. k_4 \leftarrow 2Z_{128} \cdot k_3, \quad 6. k_3 \leftarrow k_3 \cdot k_4, \quad 7. k_5 \leftarrow -k_1 \cdot k_4, \quad 8. k_1 \leftarrow k_1^2, \\
& 9. k_1 \leftarrow k_1 \cdot X_{128}, \quad 10. k_0 \leftarrow X_{22} - k_0, \quad 11. k_4 \leftarrow x_P \cdot X_{22}, \quad 12. k_4 \leftarrow Z_{22} - k_4, \\
& 13. k_6 \leftarrow k_4 \cdot k_0, \quad 14. k_6 \leftarrow k_6 \cdot Z_{128}, \quad 15. k_6 \leftarrow k_6 - k_1, \quad 16. k_6 \leftarrow k_6 - k_3, \\
& 17. k_2 \leftarrow k_2 \cdot k_5, \quad 18. k_5 \leftarrow 2k_5 y_{\psi_3^{-1}(Q)}, \quad 19. k_1 \leftarrow 2x_{\psi_3^{-1}(Q)}, \quad 20. k_3 \leftarrow k_1 + x_P, \\
& 21. k_4 \leftarrow k_1 - x_P, \quad 22. k_3 \leftarrow k_3 \cdot k_6, \quad 23. k_4 \leftarrow k_4 \cdot k_6, \quad 24. k_6 \leftarrow \alpha \cdot k_5,
\end{aligned} \tag{15}$$

where α is the parameter in $\psi_3 : (x, y) \mapsto (-x, \alpha y)$, a non-trivial square root of unity. Then $l_{[2^{128}]P, [2^{22}]P}(Q) = k_2 - k_3 + k_5 \cdot u$ and $l_{[2^{128}]P, [2^{22}]P}(\hat{\psi}_3(Q)) = k_2 + k_4 - k_6 \cdot u$.

4.4.3 Omega pairing on NSS-1536/NSS-1536($a = -3$)

According to the analysis in [45], the computational cost of using Miller's algorithm to compute the Omega pairing on NSS-1536($a = -3$) in Jacobian projective coordinates is $14\mathbf{m} + 10\mathbf{s} + 2\mathbf{M} + 1\mathbf{S}$ when the bit is 0, and $36\mathbf{m} + 16\mathbf{s} + 4\mathbf{M} + 1\mathbf{S}$ when the bit is 1. Therefore, the total cost is $127 \times (14\mathbf{m} + 10\mathbf{s} + 2\mathbf{M} + 1\mathbf{S}) + 1 \times (36\mathbf{m} + 16\mathbf{s} + 4\mathbf{M} + 1\mathbf{S}) = 1814\mathbf{m} + 1286\mathbf{s} + 258\mathbf{M} + 128\mathbf{S}$, which is equivalent to $4130\mathbf{m}$.

Table 5 Computational costs of detailed procedures for the Omega pairing by executing Algorithm 1 twice on NSS-1536

Procedure	Cost	Times
$X(P + Q)$ and $Z(P + Q)$	$3\mathbf{m} + 1\mathbf{s}$	1
<code>xDBLADD</code>	$8\mathbf{m} + 2\mathbf{s} + 2\mathbf{S}$	254
<code>xDBLADD_last</code>	$6\mathbf{m} + 2\mathbf{s}$	2
$l_{[2^{128}]P, [2^{22}]P}(Q)$ and $l_{[2^{128}]Q, [2^{22}]Q}(P)$	$26\mathbf{m} + 2\mathbf{s}$	1
Final multiplications*	$5\mathbf{M}$	1
Total cost: $2073\mathbf{m} + 515\mathbf{s} + 5\mathbf{M} + 508\mathbf{S}$ (equivalent to $3619\mathbf{m}$)		

* Final multiplications involve the expression $\widetilde{Z([2^{128}]P + Q)} \cdot \widetilde{Z([2^{22}]P + Q)} \cdot \widetilde{l_{[2^{128}]P, [2^{22}]P}(Q)} \cdot \widetilde{Z([2^{128}]Q + P)} \cdot \widetilde{Z([2^{22}]Q + P)} \cdot \widetilde{l_{[2^{128}]Q, [2^{22}]Q}(P)}$.

The computation of the Omega pairing on NSS-1536 using our algorithm is detailed in Table 5. Each of the two line functions requires $13\mathbf{m} + 1\mathbf{s}$. The calculation process of the first line function is shown in Eq. (15), and that of the second line function is similar with it.

5 Conclusion

In this work, we have proposed a novel ladder for cubical pairing computation. Our new ladder has demonstrated superior performance compared with the cubical ladder [35, Algorithm 4.2], cubical double-and-add ladder [35, Algorithm 4.1], and double-and-add ladder with compatible addition [27, Algorithm 2] when computing pairings on curves with embedding degree 2.

We have constructed a non-supersingular curve NSS-1536, and implemented both our algorithm and Miller’s algorithm on supersingular and non-supersingular curves with 1536-bit p using the C-based RELIC library. Our ladder has been implemented on SSmont-1536 and NSS-1536, rather than on SS-1536 and NSS-1536(a=-3), which are used by Miller’s algorithm, as explained in Section 4.4.

Finally, we have compared the efficiency of our method with Miller’s algorithm, evaluating both the number of $\mathbb{F}_p$ -multiplications and CPU clock cycles. The results are presented in Table 2. We have omitted the final exponentiation, as it has not been modified in our work. Our code was compiled using GCC 11.4.0 with the -O2 optimization flag and benchmarked on an Intel Core i9-12900K processor, clocked at 3.20 GHz, with TurboBoost and hyper-threading disabled. The experimental results demonstrate that our new algorithms achieve a 17.53% reduction in $\mathbb{F}_p$ -multiplications and a 13.58% reduction in CPU clock cycles for the reduced Tate pairing on SS-1536, compared with Miller’s algorithm. For the Omega pairing on NSS-1536, our methods result in a 12.37% reduction in $\mathbb{F}_p$ -multiplications and an 8.39% reduction in CPU clock cycles. For the variant of the Tate pairing, our algorithm on NSS-1536 performs on par with Miller’s algorithm on NSS-1536(a=-3).

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 12441107), Guangdong Basic and Applied Basic Research Foundation (No. 2025A1515011764), Guangdong Major Project of Basic and Applied Basic Research (No. 2019B030302008) and Guangdong Provincial Key Laboratory of Information Security Technology (No. 2023B1212060026).

References

- [1] D. F. Aranha, C. P. L. Gouvêa, T. Markmann, R. S. Wahby, and K. Liao. RELIC is an Efficient Library for Cryptography. <https://github.com/relic-toolkit/relic>.
- [2] Diego F Aranha, Youssef El Housni, and Aurore Guillevic. A survey of elliptic curves for proof systems. *Designs, Codes and Cryptography*, 91(11):3333–3378, 2023.
- [3] Paulo S. L. M. Barreto, Hae Y. Kim, Ben Lynn, and Michael Scott. Efficient algorithms for pairing-based cryptosystems. In Moti Yung, editor, *Advances in Cryptology — CRYPTO 2002*, pages 354–369, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.

- [4] Paulo SLM Barreto, Steven D Galbraith, Colm Ó' hÉigeartaigh, and Michael Scott. Efficient pairing computation on supersingular abelian varieties. *Designs, Codes and Cryptography*, 42:239–271, 2007.
- [5] Dan Boneh and Matthew Franklin. Identity-based encryption from the weil pairing. *SIAM Journal on Computing*, 32(3):586–615, 2003.
- [6] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In Colin Boyd, editor, *Advances in Cryptology — ASIACRYPT 2001*, pages 514–532, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [7] Éric Brier and Marc Joye. Weierstraß elliptic curves and side-channel attacks. In David Naccache and Pascal Paillier, editors, *Public Key Cryptography*, pages 335–345, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- [8] Shiping Cai, Zhi Hu, Zheng-An Yao, and Chang-An Zhao. The elliptic net algorithm revisited. *Journal of Cryptographic Engineering*, 14(1):43–55, 2024.
- [9] Wouter Castryck and Thomas Decru. Csidh on the surface. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography*, pages 111–129, Cham, 2020. Springer International Publishing.
- [10] Binglong Chen and Chang-An Zhao. An improvement of the elliptic net algorithm. *IEEE Transactions on Computers*, 65(9):2903–2909, 2015.
- [11] Yu Chen, Qiang Tang, and Yuyu Wang. Hierarchical integrated signature and encryption. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 514–543, Cham, 2021. Springer International Publishing.
- [12] Clifford Cocks and RGE Pinch. Identity-based cryptosystems based on the weil pairing. *Unpublished manuscript*, 170, 2001.
- [13] Yu Dai, Debiao He, Cong Peng, Zhijian Yang, and Chang-an Zhao. Revisiting pairing-friendly curves with embedding degrees 10 and 14. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 454–485, Singapore, 2025. Springer Nature Singapore.
- [14] Yu Dai, Fangguo Zhang, and Chang-an Zhao. Don't Forget Pairing-Friendly Curves with Odd Prime Embedding Degrees. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):393–419, Aug. 2023.
- [15] Iwan Duursma and Hyang-Sook Lee. Tate pairing implementation for hyperelliptic curves $y^2 = xp - x + d$. In Chi-Sung Lai, editor, *Advances in Cryptology - ASIACRYPT 2003*, pages 111–123, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [16] Youssef El Housni and Aurore Guillevic. Optimized and secure pairing-friendly elliptic curves suitable for one layer proof composition. In Stephan Krenn, Haya Shulman, and Serge Vaudenay, editors, *Cryptology and Network Security*, pages 259–279, Cham, 2020. Springer International Publishing.
- [17] Youssef El Housni and Aurore Guillevic. Families of snark-friendly 2-chains of elliptic curves. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 367–396, Cham, 2022. Springer International Publishing.
- [18] Nadia El Mrabet and Marc Joye. *Guide to pairing-based cryptography*. CRC Press, 2017.

- [19] Emmanuel Fouotsa, Laurian Azebaze Guimagang, and Raoul Ayissi. x-superoptimal pairings on elliptic curves with odd prime embedding degrees: BW 13-P 310 and BW 19-P 286. *Applicable Algebra in Engineering, Communication and Computing*, pages 1–19, 2023.
- [20] David Freeman, Michael Scott, and Edlyn Teske. A taxonomy of pairing-friendly elliptic curves. *Journal of cryptology*, 23:224–280, 2010.
- [21] Steven D Galbraith and Xibin Lin. Computing pairings using x-coordinates only. *Designs, Codes and Cryptography*, 50(3):305–324, 2009.
- [22] Robert P. Gallant, Robert J. Lambert, and Scott A. Vanstone. "Faster Point Multiplication on Elliptic Curves with Efficient Endomorphisms". In Joe Kilian, editor, *Advances in Cryptology — CRYPTO 2001*, pages 190–200, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [23] R Granger and N. P. Smart. On computing products of pairings. Cryptology ePrint Archive, Paper 2006/172, 2006.
- [24] Alexandre Grothendieck, editor. *Groupes de Monodromie en Géométrie Algébrique: SGA 7*, volume 288 of *Lecture Notes in Mathematics*. Springer-Verlag, Berlin, Heidelberg, 1972. Séminaire de Géométrie Algébrique du Bois Marie 1967–1969.
- [25] F. Hess, N.P. Smart, and F. Vercauteren. The Eta Pairing Revisited. *IEEE Transactions on Information Theory*, 52(10):4595–4602, 2006.
- [26] Eunjeong Lee, Hyang-Sook Lee, and Cheol-Min Park. Efficient and generalized pairing computation on abelian varieties. *IEEE Transactions on Information Theory*, 55(4):1793–1803, 2009.
- [27] Jianming Lin, Damien Robert, Chang-An Zhao, and Yuhao Zheng. Biextensions in pairing-based cryptography. Cryptology ePrint Archive, Paper 2025/670, 2025.
- [28] Jianming Lin, Chang-An Zhao, and Yuhao Zheng. Efficient implementation of super-optimal pairings on curves with small prime fields at the 192-bit security level. Cryptology ePrint Archive, Paper 2024/1195, 2024.
- [29] Seiichi Matsuda, Naoki Kanayama, Florian Hess, and Eiji Okamoto. Optimised versions of the ate and twisted ate pairings. In Steven D. Galbraith, editor, *Cryptography and Coding*, pages 302–312, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [30] Alfred J Menezes. *Elliptic curve public key cryptosystems*, volume 234. Springer Science & Business Media, 1993.
- [31] Victor S Miller. The weil pairing, and its efficient calculation. *Journal of cryptology*, 17:235–261, 2004.
- [32] David Mumford. Bi-extensions of formal groups. *Algebraic geometry*, (307-322), 1969.
- [33] Giacomo Pope, Krijn Reijnders, Damien Robert, Alessandro Sferlazza, and Benjamin Smith. Simpler and faster pairings from the montgomery ladder. Cryptology ePrint Archive, Paper 2025/672, 2025.
- [34] Yan Feng Qi, Chun Ming Tang, Baoan Guo, and Mao Zhi Xu. Super-optimal pairings. *Applied Mechanics and Materials*, 281:127–133, 2013.
- [35] Damien Robert. Fast pairings via biextensions and cubical arithmetic. Cryptology ePrint Archive, Paper 2024/517, 2024.

- [36] Yumi Sakemi, Tetsutaro Kobayashi, Tsunekazu Saito, and Riad S. Wahby. Pairing-Friendly Curves. Internet-Draft draft-irtf-cfrg-pairing-friendly-curves-11, Internet Engineering Task Force, November 2022. Work in Progress.
- [37] Michael Scott. Computing the tate pairing. In Alfred Menezes, editor, *Topics in Cryptology – CT-RSA 2005*, pages 293–304, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [38] Michael Scott. Faster pairings using an elliptic curve with an efficient endomorphism. In Subhamoy Maitra, C. E. Veni Madhavan, and Ramarathnam Venkatesan, editors, *Progress in Cryptology - INDOCRYPT 2005*, pages 258–269, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [39] Michael Scott and Paulo S. L. M. Barreto. Compressed pairings. In Matt Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, pages 140–156, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [40] Igor Semaev. Summation polynomials and the discrete logarithm problem on elliptic curves. Cryptology ePrint Archive, Paper 2004/031, 2004.
- [41] Katherine E Stange. The tate pairing via elliptic nets. In *Pairing-Based Cryptography–Pairing 2007: First International Conference, Tokyo, Japan, July 2-4, 2007. Proceedings 1*, pages 329–348. Springer, 2007.
- [42] Katherine E. Stange. *Elliptic nets and elliptic curves*. PhD thesis, Brown University, 2008.
- [43] Frederik Vercauteren. Optimal Pairings. *IEEE Transactions on Information Theory*, 56(1):455–461, Jan 2010.
- [44] Lawrence C Washington. *Elliptic curves: number theory and cryptography*. Chapman and Hall/CRC, 2008.
- [45] Chang-An Zhao, Dongqing Xie, Fangguo Zhang, Jingwei Zhang, and Bing-Long Chen. Computing bilinear pairings on elliptic curves with automorphisms. *Des. Codes Cryptography*, 58(1):35–44, jan 2011.
- [46] Chang-An Zhao, Fangguo Zhang, and Jiwu Huang. Speeding up the bilinear pairings computation on curves with automorphisms. Cryptology ePrint Archive, Paper 2006/474, 2006.
- [47] Chang-An Zhao, Fangguo Zhang, and Jiwu Huang. A note on the ate pairing. *International Journal of Information Security*, 7(6):379–382, 2008.

Appendix A Parameters for NSS-1536 and NSS-1536(a=-3)

The parameters for NSS-1536 are:

$$\lambda = 2^{128} + 2^{22}$$

$$r = \lambda^2 + 1$$

$$t = 2195592141961267171714702962365960662917970707291717689141872216858552941683521836440489382463572010863044615941173916628348506317139972219931556239544703890778550380978775262997834365145198686253807221365530093162704367527710098002$$

$$p = 1205156213460516294290058303014157056456046623972844475679837519532628695795901600334542512053673024831724383140444002393931208489397479162484806493945391116304932389183448695353036402348734884042660265150802503649138763160876804849839799593877672635661669161182936131275976973319194496770869056255678442234839964643913596417445111171727489512084387926530077831392990469472017004960816607631160776514660233868301869597147117074903640681165739198037712909885003957.$$

The parameters for NSS-1536(a=-3) are:

$$\lambda = 2^{128} + 2^{22}$$

$$r = \lambda^2 + 1$$

$$t = 2195592141961267171714702962365960662917970707291717689141872216858552941683521836440489382463572010863044615941173916628348506317139972219931556239544718017413437333554616938658005425418205222212190640492564683533632292453613765324$$

$$p = 1205156213460516294290058303014157056456046623972844475679837519532628695795901600334542512053673024831724383140444002393931208489397479162484806493945406624469207463668109353700503914594024014948044281816422726309738854305765617614667566529983577439786830115200587386546655257151035764894472675600569259980604716356469015993008938770523595662930865187596430653844453385201276594927780119532470875713884979181168011014379526948138369682311565798430618983567980413.$$

Appendix B Related Algorithms

In this appendix, we present some associated algorithms required for pairing computation via biextension $X_{2(\mathcal{O}_E)}$ and cubical arithmetic on curves $y^2 = x^3 + ax$, including the case where $a = 1$.

Algorithm 3 xDBL: Point doubling on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + ax$

Input: Point $P = (X_P : Z_P)$, where $P \in E/\langle \pm 1 \rangle$ and $X_P, Z_P \in \mathbb{F}_p$.

Output: Point $[2]P = (X_{[2]P} : Z_{[2]P})$.

```

1:  $t_1 \leftarrow X_P^2$  ▷ 1s
2:  $t_2 \leftarrow Z_P^2$  ▷ 1s
3:  $t_3 \leftarrow a \cdot t_2$ 
4:  $X_{[2]P} \leftarrow (t_1 - t_3)^2$  ▷ 1s
5:  $t_4 \leftarrow 4X_P \cdot Z_P$  ▷ 1m
6:  $Z_{[2]P} \leftarrow t_4 \cdot (t_1 + t_3)$  ▷ 1m
7: return  $X_{[2]P}, Z_{[2]P}$  ▷ Total cost: 2m + 3s

```

Algorithm 4 xADD: Differential addition on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + ax$

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$ and $Q = (x_Q, 1)$, where $P, Q, Q + P \in E/\langle \pm 1 \rangle$, $X_P, Z_P, x_Q \in \mathbb{F}_p$ and $X_{Q+P}, Z_{Q+P} \in \mathbb{F}_{p^2}$.

Output: Point $Q + [2]P = (X_{Q+[2]P} : Z_{Q+[2]P})$.

```

1:  $X_P Z_{Q+P} \leftarrow X_P \cdot Z_{Q+P}$  ▷ 2m
2:  $X_{Q+P} Z_P \leftarrow X_{Q+P} \cdot Z_P$  ▷ 2m
3:  $b \leftarrow (X_P + Z_P) \cdot (X_{Q+P} - a \cdot Z_{Q+P}) - X_{Q+P} Z_P + a \cdot X_P Z_{Q+P}$  ▷ 2m
4:  $c \leftarrow b^2$  ▷ 1S
5:  $d \leftarrow (X_P Z_{Q+P} - X_{Q+P} Z_P)^2$  ▷ 1S
6:  $Z_{Q+[2]P} \leftarrow d \cdot x_Q$  ▷ 2m
7:  $X_{Q+[2]P} \leftarrow c \cdot z_Q$ 
8: return  $X_{Q+[2]P}, Z_{Q+[2]P}$  ▷ Total cost: 8m + 2S

```

Algorithm 5 xDBL(a=1): Point doubling on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + x$

Input: Point $P = (X_P : Z_P)$, where $P \in E/\langle \pm 1 \rangle$ and $X_P, Z_P \in \mathbb{F}_p$.

Output: Point $[2]P = (X_{[2]P} : Z_{[2]P})$.

```

1:  $t_0 \leftarrow X_{[2]P} - Z_{[2]P}$ 
2:  $t_1 \leftarrow X_{[2]P} + Z_{[2]P}$ 
3:  $t_0 \leftarrow t_0^2$  ▷ 1s
4:  $t_1 \leftarrow t_1^2$  ▷ 1s
5:  $Z_{[2]P} \leftarrow t_0$ 
6:  $X_{[2]P} \leftarrow t_1 \cdot Z_{[2]P}$  ▷ 1m
7:  $t_1 \leftarrow t_1 - t_0$ 
8:  $t_0 \leftarrow 2 \cdot t_1$ 
9:  $Z_{[2]P} \leftarrow Z_{[2]P} + t_0$ 
10:  $Z_{[2]P} \leftarrow Z_{[2]P} \cdot t_1$  ▷ 1m
11: return  $X_{[2]P}, Z_{[2]P}$  ▷ Total cost: 2m + 2s

```

Algorithm 6 xADD(a=1): Differential addition on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + x$

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$ and $Q = (x_Q, 1)$, where $P, Q, Q + P \in E/\langle \pm 1 \rangle$, $X_P, Z_P, x_Q \in \mathbb{F}_p$ and $X_{Q+P}, Z_{Q+P} \in \mathbb{F}_{p^2}$.

Output: Point $Q + [2]P = (X_{Q+[2]P} : Z_{Q+[2]P})$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_1 \leftarrow X_P - Z_P$ 
3:  $t_2 \leftarrow X_{Q+P} - Z_{Q+P}$ 
4:  $X_{Q+[2]P} \leftarrow X_{Q+P} + Z_{Q+P}$ 
5:  $t_0 \leftarrow t_0 \cdot t_2$  ▷ 2m
6:  $t_1 \leftarrow t_1 \cdot X_{Q+[2]P}$  ▷ 2m
7:  $Z_{Q+[2]P} \leftarrow t_0 - t_1$ 
8:  $X_{Q+[2]P} \leftarrow t_0 + t_1$ 
9:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P}^2$  ▷ 1S
10:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P}^2$  ▷ 1S
11:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P} \cdot x_Q$  ▷ 2m
12:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P} \cdot z_Q$ 
13: return  $X_{Q+[2]P}, Z_{Q+[2]P}$  ▷ Total cost: 6m + 2S

```

Algorithm 7 xDBLADD: Double-and-add on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + x$

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$ and $Q = (x_Q, 1)$, where $P, Q, Q + P \in E/\langle \pm 1 \rangle$, $X_P, Z_P, x_Q \in \mathbb{F}_p$ and $X_{Q+P}, Z_{Q+P} \in \mathbb{F}_{p^2}$.

Output: Points $P = (X_{[2]P} : Z_{[2]P})$ and $Q + [2]P = (X_{Q+[2]P} : Z_{Q+[2]P})$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_1 \leftarrow X_P - Z_P$ 
3:  $X_{[2]P} \leftarrow t_0^2$  ▷ 1s
4:  $t_2 \leftarrow X_{Q+P} - Z_{Q+P}$ 
5:  $X_{Q+[2]P} \leftarrow X_{Q+P} + Z_{Q+P}$ 
6:  $t_0 \leftarrow t_0 \cdot t_2$  ▷ 2m
7:  $Z_{[2]P} \leftarrow t_1^2$  ▷ 1s
8:  $Z2_{[2]P} \leftarrow 2 \cdot Z_{[2]P}$ 
9:  $t_1 \leftarrow t_1 \cdot X_{Q+[2]P}$  ▷ 2m
10:  $t_2 \leftarrow X_{[2]P} - Z_{[2]P}$ 
11:  $X_{[2]P} \leftarrow X_{[2]P} \cdot Z2_{[2]P}$  ▷ 1m
12:  $X_{Q+[2]P} \leftarrow t_2$ 
13:  $Z_{Q+[2]P} \leftarrow t_0 - t_1$ 
14:  $Z_{[2]P} \leftarrow X_{Q+[2]P} + Z2_{[2]P}$ 
15:  $X_{Q+[2]P} \leftarrow t_0 + t_1$ 
16:  $Z_{[2]P} \leftarrow Z_{[2]P} \cdot t_2$  ▷ 1m
17:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P}^2$  ▷ 1S
18:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P}^2$  ▷ 1S
19:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P} \cdot x_Q$  ▷ 2m
20:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P} \cdot z_Q$ 
21: return  $(X_{[2]P} : Z_{[2]P}), (X_{Q+[2]P} : Z_{Q+[2]P})$  ▷ Total cost: 8m + 2s + 2S

```

Algorithm 8 Cubical ladder

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$, $Q = (x_Q : 1)$ where $x_Q \in \mathbb{F}_p$, and ℓ in binary $\ell = (\ell_{n-1}\ell_{n-2} \cdots \ell_0)_2$.

Output: The Miller function $f_{\ell,P}(Q)$.

```
1:  $(X_{[\ell]P}, Z_{[\ell]P}) \leftarrow (X_P, Z_P)$ 
2:  $(X_{[\ell]PP}, Z_{[\ell]PP}) \leftarrow \text{xDBL}(X_P, Z_P)$ 
3:  $(X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow \text{xADD}(X_{Q+P}, Z_{Q+P}, X_P, Z_P, x_Q, 1)$ 
4: for  $i \leftarrow n - 2$  to 0 do ▷ Process bits left to right
5:   if  $\ell_i = 0$  then
6:      $(X_{Q+[\ell]PP}, Z_{Q+[\ell]PP}) \leftarrow \text{xADD}(X_{Q+[\ell]PP}, Z_{Q+[\ell]PP}, X_{[\ell]P}, Z_{[\ell]P}, 1, \frac{Z_{Q+P}}{X_{Q+P}})$ 
7:      $(X_{[\ell]PP}, Z_{[\ell]PP}) \leftarrow \text{xADD}(X_{[\ell]PP}, Z_{[\ell]PP}, X_{[\ell]P}, Z_{[\ell]P}, x_P, z_P)$ 
8:      $(X_{[\ell]P}, Z_{[\ell]P}) \leftarrow \text{xDBL}(X_{[\ell]P}, Z_{[\ell]P})$ 
9:   else if  $\ell_i = 1$  then
10:     $(X_{Q+[\ell]PP}, Z_{Q+[\ell]PP}) \leftarrow \text{xADD}(X_{Q+[\ell]PP}, Z_{Q+[\ell]PP}, X_{[\ell]PP}, Z_{[\ell]PP}, x_Q, 1)$ 
11:     $(X_{[\ell]P}, Z_{[\ell]P}) \leftarrow \text{xADD}(X_{[\ell]PP}, Z_{[\ell]PP}, X_{[\ell]P}, Z_{[\ell]P}, x_P, z_P)$ 
12:     $(X_{[\ell]PP}, Z_{[\ell]PP}) \leftarrow \text{xDBL}(X_{[\ell]PP}, Z_{[\ell]PP})$ 
13:   end if
14: end for
15: return  $Z_{Q+[\ell]PP} \cdot \overline{X_{[\ell]P}}$ 
```

Algorithm 9 Double-and-add ladder

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$, $Q = (x_Q : 1)$ where $x_Q \in \mathbb{F}_p$, and ℓ in binary $\ell = (\ell_{n-1}\ell_{n-2} \cdots \ell_0)_2$.

Output: The Miller function $f_{\ell,P}(Q)$.

```
1:  $(X_{[\ell]P}, Z_{[\ell]P}, X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow (X_P, Z_P, X_{Q+P}, Z_{Q+P})$ 
2: for  $i \leftarrow n - 2$  to 0 do ▷ Process bits left to right
3:    $(X_{[\ell]P}, Z_{[\ell]P}, X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow$ 
      $\text{xDBLADD}(X_{[\ell]P}, Z_{[\ell]P}, X_{Q+[\ell]P}, Z_{Q+[\ell]P}, x_Q, 1)$ 
4:   if  $\ell_i = 1$  then
5:      $(X_{[\ell]P}, Z_{[\ell]P}, X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow$ 
        $\text{ThreeWayAdd}(X_{[\ell]P}, Z_{[\ell]P}, X_P, Z_P, x_Q, 1, X_{Q+[\ell]P}, Z_{Q+[\ell]P}, X_{Q+P}, Z_{Q+P})$ 
6:   end if
7: end for
8: return  $Z_{[\ell]P+Q} \cdot \overline{X_{[\ell]P}}$ 
```

Algorithm 10 cADD: Compatible addition on $E/\langle \pm 1 \rangle$, , where $E/\mathbb{F}_p : y^2 = x^3 + ax$

Input: Points $P_1 = (X_{P_1} : 1)$, $P_2 = (X_{P_2} : Z_{P_2})$, $P_1 - Q = (X_{P_1-Q} : Z_{P_1-Q})$, $P_2 + Q = (X_{P_2+Q} : Z_{P_2+Q})$, where $P_1, P_2 \in E(\mathbb{F}_p)/\langle \pm 1 \rangle$ and $P_2 + Q \in E(\mathbb{F}_{p^2})/\langle \pm 1 \rangle$, with $X_{P_1-Q} \in \mathbb{F}_{p^2}$ and $Z_{P_1-Q} \in \mathbb{F}_p$

Output: The real parts of coordinates $(X_{P_1+P_2} : Z_{P_1+P_2})$

```

1:  $t_1 \leftarrow X_{P_1} \cdot X_{P_2}$  ▷ 1m
2:  $t_2 \leftarrow X_{P_1} \cdot Z_{P_2}$  ▷ 1m
3:  $t_3 \leftarrow (t_1 - Z_{P_2})^2$  ▷ 1s
4:  $t_{33} \leftarrow (t_2 - X_{P_2})^2$  ▷ 1s
5:  $t_4 \leftarrow 2 \cdot (t_1 + Z_{P_2}) \cdot (t_2 + X_{P_2})$  ▷ 1m
6:  $t_5 \leftarrow X_{P_1-Q} \cdot X_{P_2+Q}$  ▷ 1M
7:  $t_6 \leftarrow X_{P_1-Q} \cdot Z_{P_2+Q}$  ▷ 1M
8:  $t_{55} \leftarrow Z_{P_2+Q} \cdot Z_{P_1-Q}$  ▷ 2m
9:  $t_{66} \leftarrow X_{P_2+Q} \cdot Z_{P_1-Q}$  ▷ 2m
10:  $t_7 \leftarrow (t_5 - t_{55})^2$  ▷ 1S
11:  $t_{77} \leftarrow (t_6 - t_{66})^2$  ▷ 1S
12:  $t_8 \leftarrow 2 \cdot (t_5 + t_{55}) \cdot (t_6 + t_{66})$  ▷ For the real part: 2m
13:  $X_{P_1+P_2} \leftarrow t_3 \cdot t_8 - t_4 \cdot t_7$  ▷ For the real part: 2m
14:  $Z_{P_1+P_2} \leftarrow t_3 \cdot t_{77} - t_{33} \cdot t_7$  ▷ For the real part: 2m
15: return  $X_{P_1+P_2}, Z_{P_1+P_2}$  ▷ Total cost: 13m + 2s + 2M + 2S

```

Algorithm 11 Double-and-add ladder with compatible addition

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$, $Q = (x_Q : 1)$ where $x_Q \in \mathbb{F}_p$, and ℓ in binary $\ell = (\ell_{n-1}\ell_{n-2} \cdots \ell_0)_2$.

Input: x -coordinates $x(P) = (X_P : 1)$, $x(Q + P) = (X_{Q+P} : Z_{Q+P})$, $x(P - Q) = (X_{P-Q} : Z_{P-Q})$, $X_P, Z_{Q+P}, Z_{P-Q} \in \mathbb{F}_p$, $X_{Q+P}, X_{P-Q} \in \mathbb{F}_{p^2} \setminus \mathbb{F}_p$, and ℓ in binary $\ell = (\ell_{n-1}\ell_{n-2} \cdots \ell_0)_2$.

Output: The Miller function $f_{\ell,P}(Q)$.

```

1:  $(X_{[\ell]P}, Z_{[\ell]P}, X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow (X_P, 1, X_{Q+P}, Z_{Q+P})$ 
2: for  $i \leftarrow n - 2$  to 0 do ▷ Process bits left to right
3:   if  $\ell_i = 0$  then
4:      $(X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow \mathbf{xADD}(X_{Q+[\ell]P}, Z_{Q+[\ell]P}, X_P, Z_P, x_Q, 1)$ 
5:      $(X_{[\ell]P}, Z_{[\ell]P}) \leftarrow \mathbf{xDBL}(X_{[\ell]P}, Z_{[\ell]P})$ 
6:   else if  $\ell_i = 1$  then
7:      $t_1, t_2 \leftarrow \mathbf{cADD}(X_P, X_{[\ell]P}, Z_{[\ell]P}, X_{P-Q}, Z_{P-Q}, X_{Q+[\ell]P}, Z_{Q+[\ell]P})$  ▷  $[\ell + 1]P$ 
8:      $(s_1, s_2) \leftarrow (N(X_{P-Q}), Z_{P-Q} \overline{X_{P-Q}})$  ▷  $N(X_{P-Q})$  is the norm of  $X_{P-Q}$ 
9:      $(X_{Q+[\ell]P}, Z_{Q+[\ell]P}) \leftarrow \mathbf{xADD}(t_1, t_2, X_{Q+[\ell]P}, Z_{Q+[\ell]P}, s_1, s_2)$  ▷ 6m + 1M + 2S
10:     $(X_{[\ell]P}, Z_{[\ell]P}) \leftarrow \mathbf{xADD}(t_1, t_2, X_{[\ell]P}, Z_{[\ell]P}, x_P, 1)$ 
11:   end if
12: end for
13: return  $Z_{Q+[\ell]P} \cdot \overline{X_{[\ell]P}}$ 

```

Algorithm 12 xDBLADD_s: Double-and-add on $E/\langle \pm 1 \rangle$, where $E/\mathbb{F}_p : y^2 = x^3 + x$

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$ and $Q = (x_Q, 1)$, where $P, Q, Q + P \in E/\langle \pm 1 \rangle$, $X_P, Z_P, x_Q \in \mathbb{F}_p$ and $X_{Q+P}, Z_{Q+P} \in \mathbb{F}_{p^2}$.

Output: Points $P = (X_{[2]P} : Z_{[2]P})$, $Q + [2]P = (X_{Q+[2]P} : Z_{Q+[2]P})$, and values $r_0 = X_P + Z_P$ and $r_1 = X_P - Z_P$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $r_0 \leftarrow t_0$ 
3:  $t_1 \leftarrow X_P - Z_P$ 
4:  $r_1 \leftarrow t_1$ 
5:  $X_{[2]P} \leftarrow t_0^2$  ▷ 1s
6:  $t_2 \leftarrow X_{Q+P} - Z_{Q+P}$ 
7:  $X_{Q+[2]P} \leftarrow X_{Q+P} + Z_{Q+P}$ 
8:  $t_0 \leftarrow t_0 \cdot t_2$  ▷ 2m
9:  $Z_{[2]P} \leftarrow t_1^2$  ▷ 1s
10:  $Z_{[2]P} \leftarrow 2 \cdot Z_{[2]P}$ 
11:  $t_1 \leftarrow t_1 \cdot X_{Q+[2]P}$  ▷ 2m
12:  $t_2 \leftarrow X_{[2]P} - Z_{[2]P}$ 
13:  $X_{[2]P} \leftarrow X_{[2]P} \cdot Z_{[2]P}$  ▷ 1m
14:  $X_{Q+[2]P} \leftarrow t_2$ 
15:  $Z_{Q+[2]P} \leftarrow t_0 - t_1$ 
16:  $Z_{[2]P} \leftarrow X_{Q+[2]P} + Z_{[2]P}$ 
17:  $X_{Q+[2]P} \leftarrow t_0 + t_1$ 
18:  $Z_{[2]P} \leftarrow Z_{[2]P} \cdot t_2$  ▷ 1m
19:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P}^2$  ▷ 1S
20:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P}^2$  ▷ 1S
21:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P} \cdot x_Q$  ▷ 2m
22:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P} \cdot z_Q$ 
23: return  $(X_{[2]P} : Z_{[2]P})$ ,  $(X_{Q+[2]P} : Z_{Q+[2]P})$ ,  $r_0$ ,  $r_1$  ▷ Total cost: 8m + 2s + 2S

```

Algorithm 13 xADD_s: Differential addition on $E/\langle \pm 1 \rangle$ with storage, where $E/\mathbb{F}_p : y^2 = x^3 + x$

Input: Points $P = (X_P : Z_P)$, $Q + P = (X_{Q+P} : Z_{Q+P})$ and $Q = (x_Q, 1)$, where $P, Q, Q + P \in E/\langle \pm 1 \rangle$, $X_P, Z_P, x_Q \in \mathbb{F}_p$ and $X_{Q+P}, Z_{Q+P} \in \mathbb{F}_{p^2}$, and values r_0 and r_1 computed in Line 2 and 4 of Algorithm 12.

Output: Point $Q + [2]P = (X_{Q+[2]P} : Z_{Q+[2]P})$.

```

1:  $t_0 \leftarrow r_0$ 
2:  $t_1 \leftarrow r_1$ 
3:  $t_2 \leftarrow X_{Q+P} - Z_{Q+P}$ 
4:  $X_{Q+[2]P} \leftarrow X_{Q+P} + Z_{Q+P}$ 
5:  $t_0 \leftarrow t_0 \cdot t_2$  ▷ 2m
6:  $t_1 \leftarrow t_1 \cdot X_{Q+[2]P}$  ▷ 2m
7:  $Z_{Q+[2]P} \leftarrow t_0 - t_1$ 
8:  $X_{Q+[2]P} \leftarrow t_0 + t_1$ 
9:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P}^2$  ▷ 1S
10:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P}^2$  ▷ 1S
11:  $Z_{Q+[2]P} \leftarrow Z_{Q+[2]P} \cdot x_Q$  ▷ 2m
12:  $X_{Q+[2]P} \leftarrow X_{Q+[2]P} \cdot z_Q$ 
13: return  $X_{Q+[2]P}, Z_{Q+[2]P}$  ▷ Total cost: 6m + 2S

```
